From Grokking to Micro-Routing: Graph Neural Cellular Automata for Sub-Millisecond Agent Tool Selection

Julian M. Kleber
Independent Researcher
julian.m.kleber@gmail.com

October 2026

Abstract

Selecting which tool to invoke in an autonomous agent is a micro-routing decision, yet current harnesses dispatch every turn through multi-billion-parameter LLMs. We introduce **Graph Neural Cellular Automata (GNCA)**, a 35.3k–59.2k parameter System-1 reflex router that selects tools via recurrent message passing on a relational trace graph. In **Lean 4**, we machine-check six structural properties of the discrete topology space with zero axioms, including star-vs-chain diameter separation ($O(1)$ vs $\Omega(V)$), conditional contractive stability, and a simulation-and-compression duality proving that K cellular steps on a tree graph exact-simulate any depth-K binary decision tree while compressing parameters by 307× relative to the 18.2M-parameter Hermes Random Forest. GNCA executes batch-1 CPU forward passes in **0.35–0.66** ms (42.1 ms for local LLMs). Across four 3-seed benchmarks on unified CUDA hardware, GNCA achieves $\mathbf{87.09 \pm 1.57}$% test top-1 on **API-Bank** (outperforming API-Bank Random Forest at 80.13 ± 0.55%, $p = 0.0002$, and HistGradientBoosting at 84.75 ± 0.59%, $p = 0.0036$), $\mathbf{76.30 \pm 1.85}$% test top-1 on **ToolBench** (vs 74.33 ± 1.12% ToolBench Random Forest), and $\mathbf{76.83 \pm 0.60}$% test top-1 on **BFCL v4** (vs 74.18 ± 0.96% BFCL Random Forest, $p = 0.0022$). On **Hermes** (3,530 test turns), 10k-step class-weighted GNCA trades top-1 (64.64%–66.19% curated test top-1) for 2.15× higher rare-tool macro-F1 (0.608 vs 0.283), while unweighted label-smoothed Tree GNCA reaches 67.37% curated test top-1 at 10k steps, 68.07% curated test top-1 over a 200k grokking progression, and $\mathbf{68.92 \pm 0.47}$% curated test top-1 across 3 seeds vs 67.88% curated test top-1 for Random Forest and 69.47% curated test top-1 for HistGradientBoosting.

1. Introduction

Autonomous software agents operate as iterative closed-loop systems: observing environment feedback (compiler diagnostics, file diffs, test logs) and synthesizing next actions. In biological nervous systems, routine motor actions do not require deliberative processing in the cerebral cortex; a low-level reflex arc in the spinal cord coordinates rapid responses in milliseconds. In contrast, modern AI agents dispatch every mundane operation through massive, multi-billion-parameter autoregressive models. This design causes critical bottlenecks:

1. **Latency and Memory Pressure.** Generating tokens sequentially causes high latency and heavy memory traffic, slowing down agent loops.
2. **Context Window Bloat.** Intermediate steps fill context windows with repetitive JSON schema and syntax clutter.
3. **Rigid 1D Assumptions.** Prior cellular models restrict computation to 1D lattices. Signals crawl one step at a time like a game of telephone, ignoring that agent execution traces form hierarchical trees.

Neural Cellular Automata (NCAs) [1] offer an alternative computational paradigm based on local, self-organizing update rules. Historically, NCAs were studied in continuous 2D grids for biological morphogenesis, texture generation, and self-repair. However, their core inductive biases (iterative spatial message passing across discrete cells governed by uniform update rules) align naturally with structured sequential decision processes. Like biological cells communicating with their immediate neighbors to form complex tissues, NCA cells update their local states in parallel without global bottlenecking. Recent investigations (e.g., Mittal et al. [5]) probed token-level text NCAs, but their application to structured multi-action tool execution has remained limited to rigid sequential chains.

Contributions. In this paper, I extend Neural Cellular Automata beyond 1D topologies into a graph framework for hierarchical agent routing (fixed star and tree topologies plus optional continuous architecture search). My core contributions are fourfold.

- **Multi-Benchmark Graph Cellular Routing.** I formulate GNCA, replacing 1D convolutional stencils with relational graph message passing across a 59.2k-parameter cellular update rule (307× smaller than an 18.2M-parameter Hermes Random Forest). Across four multi-seed benchmarks, GNCA achieves $\mathbf{87.09 \pm 1.57}$% test top-1 on **API-Bank** (+6.97 pp over API-Bank Random Forest, $p = 0.0002$; +2.35 pp over HistGradientBoosting, $p = 0.0036$), $\mathbf{76.30 \pm 1.85}$% test top-1 on **ToolBench** (+1.98 pp

over ToolBench Random Forest), $\mathbf{76.83 \pm 0.60}\%$ test top-1 on **BFCL v4** (+2.65 pp over BFCL Random Forest, $p = 0.0022$), and $\mathbf{68.92 \pm 0.47}\%$ curated test top-1 in 3-seed unweighted label-smoothed evaluation on **Hermes** (68.07% curated test top-1 over the 200k grokking schedule; 66.19% curated test top-1 and 2.15× higher rare-tool macro-F1 under 10k class weighting).

- **Machine-Checked Structural Proofs, Tree–NCA Duality, & Contractive Bridge.** In Lean 4, I formally verify 1D lattice subsumption, tree subsumption, permutation equivariance, and star-vs-chain diameter separation ($O(1)$ vs $\Omega(V)$) with zero unproven axioms. Furthermore, under the assumption of threshold-gated tree transitions, I formally prove a conditional simulation and parameter-compression duality between binary decision trees and tree cellular automata (Theorem 6), explaining why tree ensembles and tree NCAs induce closely aligned partitions while weight-tied cellular rules achieve exponential $\mathcal{O}(M(2^K - 1))$ parameter compression. I complement these structural proofs with a post-hoc top-$k = 2$ discretization audit of meta-learned α matrices and a 3-seed spectrally normalized contractive training study enforcing Theorem 4's conditional dyadic contraction premise ($\rho = 0.427 < 0.50$ on 100% of test turns).
- **Dual-System Architecture & Calibrated Risk-Coverage Routing.** I introduce a System-1/System-2 agent harness with risk-gated dynamic schema pruning, combining normalized prediction entropy with the node-mean root-mean-square cellular step norm. Across 3,530 held-out test turns (3 seeds), the composite uncertainty score achieves 0.730 ± 0.008 error-detection AUROC and increases retained System-1 accuracy from 69.10% to 90.48% along the selective risk-coverage frontier.
- **Multi-Seed Factorial Ablations & Controlled Router Diagnostics.** I conduct a 3-seed factorial ablation on unified CUDA hardware isolating recurrence depth $K \in \{2, 4, 8\}$, loss weighting (class-weighted vs label-smoothed), topology, cross-turn carry, and dynamic halting, alongside matched-actuator router ablations on $N = 692$ BFCL v4 test turns and exploratory end-to-end pilot diagnostics.

2. Related Work

Neural Cellular Automata and Reasoning. Mordvintsev et al. [1] introduced Growing Neural Cellular Automata, demonstrating self-organizing pattern morphogenesis and damage recovery. Subsequent work extended NCAs to multi-scale hierarchical structures [2] and decentralized algorithmic reasoning [4]. Grattarola et al. [3] introduced Learning Graph Cellular Automata (LGCA), showing that GNNs can execute cellular update rules over fixed, predefined graphs (e.g., Voronoi meshes). In language representation, Mittal et al. [5] and Lee et al. [6] demonstrated that cellular rules can learn 1D token representations. My framework departs from token generation and static GCA. Because agent traces lack a predefined graph, I introduce hand-crafted broadcast topologies and continuous architecture search. I combine these with cross-turn carry and Lean 4 proofs of structural invariants.

Tool Calling, Agent Harnesses, and Pre-Routing. Tool use in autonomous systems is evaluated on benchmarks like the Berkeley Function Calling Leaderboard (BFCL) [7] and SWE-bench. Production agent harnesses execute many typed System-1 decisions per task [10]. Singla et al. [9] showed that pre-inference action routing cuts prompt latency and token overhead by half. Expanding on these findings, real agents must handle three coupled tasks, namely detecting irrelevance to abstain gracefully, binding argument slots to workspace entities, and coordinating multi-turn plans.

Architecture Meta-Learning & Graph Bottlenecks. Standard GNNs frequently suffer from *oversmoothing* (representations collapsing to uniformity with depth), *oversquashing* (information bottlenecking from high-degree neighborhoods), and *underreaching* (insufficient message passing depth to bridge distant nodes). By casting topology selection as bilevel architecture meta-learning (optional) or fixed star/tree wiring, the framework targets sparse relational graphs that balance local message passing with long-range goal broadcasting.

3. Task Formulation & Architecture

3.1. Real-World Operational Context & Action Formulation

In real-world software engineering agents (e.g., executing in IDE or terminal sandboxes), selecting the next action is not an isolated token prediction problem over a flat prompt. Instead, the agent operates in an interactive closed loop situated within an operational context state $\mathbf{s}_t$.

$$\mathbf{s}_t = (\mathbf{g}_{\text{intent}},\ \mathbf{w}_{\text{env}},\ \tau_{\text{hist}},\ \mathcal{T}_{\text{reg}}) \in \mathcal{X}_{\text{ctx}} \tag{1}$$

The four context channels capture the heterogeneous dependencies of practical workflows.

1. **High-Level Intent ($\mathbf{g}_{\text{intent}}$).** The user goal, issue specification, or immediate task turn defining the overarching objective.
2. **Workspace & Environment State ($\mathbf{w}_{\text{env}}$).** File paths, working directory depth, git branch/clean status, compiler and test exit codes, and runtime error diagnostics.
3. **Causal Trajectory History (τ_{hist}).** Chronological sequence of prior tool calls and execution feedback $\tau_{\text{hist}} = \{(u_{t-k}, r_{t-k})\}_{k=1}^{L-1}$ over an L-step causal window. Here $u_{t-k} \in \mathcal{V}$ is the invoked tool and r_{t-k} contains execution diagnostics (output length, error flag, stdout/stderr status).
4. **Tool Registry & Schema Constraints ($\mathcal{T}_{\text{reg}}$).** Candidate tool signatures $|\mathcal{V}|$, argument constraints,

and permission boundaries (read-only introspection vs state-mutating shell execution).

From this context $\mathbf{s}_t$, the agent synthesizes a four-part action tuple.

$$\mathbf{a}_t = (y_{\text{tool}},\ \mathbf{y}_{\text{arg}},\ g_{\text{abstain}},\ \tau_{\text{escalate}}) \tag{2}$$

Here $y_{\text{tool}} \in \mathcal{V}$ is the discrete tool identifier chosen from vocabulary $|\mathcal{V}| = 13$ (e.g., file reading, file editing, shell command execution, directory listing, and conversational response). The vector $\mathbf{y}_{\text{arg}} \in \mathcal{A}$ predicts structured argument slots mapping workspace entities. The calibrated scalar gate $g_{\text{abstain}} \in [0, 1]$ indicates whether the agent should abstain from external tool execution. Finally, $\tau_{\text{escalate}} \in [0, 1]$ provides a predictive execution risk score governing System-2 LLM fallback.

3.2. Beyond 1D: Graph Neural Cellular Automata

Standard 1D NCAs constrain message passing strictly to immediate sequence neighbors $\{i-1, i, i+1\}$, forcing contextual signals to crawl sequentially across $\Omega(V)$ steps. In agent workflows, however, the root goal and environmental diagnostics must directly modulate every historical turn, while recursive tool subtrees branch hierarchically.

Topology of Employed Algorithms. Figure 1 illustrates the spatial information-flow topology of each algorithm evaluated in this work. In 1D lattices (Figure 1a), messages crawl along a chain, creating an information bottleneck where the goal cell at index 0 requires $V-1 = 4$ cellular steps to reach the most recent action turn $t-1$. In contrast, Star GNCA (Figure 1b) introduces a centralized broadcast hub, collapsing the goal-to-cell diameter to 1 hop for all nodes. Tree GNCA (Figure 1c) organizes the causal history into a hierarchical call-tree DAG where recent execution turns $(t-2, t-1)$ are siblings subordinate to earlier subtask actions $(t-4)$, reflecting the branching structure of real agent execution trees. Meta-Learned GNCA (Figure 1d) treats edge weights as continuous differentiable parameters $\alpha \in \mathbb{R}^{5\times5}$, converging to a sparse directed tree backbone. In comparison, non-cellular baselines lack spatial recurrence: the Flat MLP (Figure 1e) flattens all cells into a monolithic 730-dimensional vector, while Random Forest (Figure 1f) and HistGradientBoosting (Figure 1g) fit large ensembles of independent decision trees. As summarized in the hop-distance matrix (Figure 1h), Star and Tree topologies ensure rapid goal signal propagation across all cells within 1–2 steps, enabling the weight-tied cellular rule to synthesize global context without parameter bloat.

I map the operational context $\mathbf{s}_t$ to an expressive relational graph $\mathcal{G} = (\mathcal{V}_{\text{graph}}, \mathcal{E}, A)$ with $|\mathcal{V}_{\text{graph}}| = V = 5$ nodes over a causal window $L = 5$.

- **Node** 0 **(Intent & Environment Hub).** Anchors the global intent $\mathbf{g}_{\text{intent}}$ concatenated with workspace diagnostic scalars $\mathbf{w}_{\text{env}}$.
- **Nodes** $1 \dots 4$ **(Causal Trajectory Cells).** Chronological execution turns (u_{t-k}, r_{t-k}), capturing both the historical operation and runtime feedback.

Each node maintains a compact 16-number scratchpad ($h_i^{(t)} \in \mathbb{R}^{16}$, initialized to $\mathbf{0}$). Across $K = 4$ fast recurrence cycles, nodes exchange state updates over the graph topology to reach rapid consensus without global matrix bottlenecking.

Continuous Spatial Perception. Let $A \in \mathbb{R}^{V\times V}$ denote a **row-stochastic** adjacency ($A_{ij} \geq 0$, $\sum_{j=1}^{V} A_{ij} = 1$ for all i). Adjacency A can be fixed or continuously parameterized via row-wise softmax(α/τ). Each cell carries a concatenated observation-hidden vector $x_i^{(t)} \in \mathbb{R}^C$ with $C = D + H$, where hidden width H=16. Stacking $x_i^{(t)}$ yields observation matrix $x^{(t)} \in \mathbb{R}^{V\times C}$. Neighbor aggregation is a left multiply by A for low-pass consensus.

$$\bar{x}_i^{(t)} = \sum_{j=1}^{V} A_{ij}\, x_j^{(t)}, \qquad \bar{\mathbf{x}}^{(t)} = A\,\mathbf{x}^{(t)}. \tag{3}$$

The high-pass channel computes the directed difference operator $(A - I)\mathbf{x} = -(I - A)\mathbf{x}$ on the same row-stochastic adjacency A.

$$\delta_i^{(t)} = \sum_{j=1}^{V} A_{ij}\big(x_j^{(t)} - x_i^{(t)}\big), \qquad \boldsymbol{\delta}^{(t)} = (A - I)\,\mathbf{x}^{(t)}. \tag{4}$$

In directed graph signal processing, $L_{\text{rw}} = I - A$ is the standard random-walk graph Laplacian for asymmetric causal routing graphs. Its row sums vanish $(\sum_j (A_{ij} - \delta_{ij}) = 0)$, so $(A-I)\mathbf{x}$ vanishes on spatially constant states. It thus isolates the directed local gradient $\bar{x}_i^{(t)} - x_i^{(t)}$ away from neighborhood consensus without requiring symmetric degree normalization. The perception tensor stacks identity, low-pass, and high-pass channels along the feature axis. This construction mirrors classical NCAs [1] on the trace graph.

$$p_i^{(t)} = \left[x_i^{(t)},\ \bar{x}_i^{(t)},\ \delta_i^{(t)}\right] \in \mathbb{R}^{3C}. \tag{5}$$

Pointwise State Update. A shared 1×1 convolution over the $3C$-channel perceived tensor computes the gated state update $\Delta h_i^{(t)} \in \mathbb{R}^H$ from $p_i^{(t)}$.

$$\begin{aligned}[\tilde{\Delta} h_i^{(t)},\ g_i^{(t)}] &= W_2\ \mathrm{ReLU}(W_1 p_i^{(t)} + b_1),\\ \Delta h_i^{(t)} &= \sigma(g_i^{(t)}) \odot \tilde{\Delta} h_i^{(t)}.\end{aligned} \tag{6}$$

Here $W_1 \in \mathbb{R}^{D_{\text{mid}}\times 3C}$, $b_1 \in \mathbb{R}^{D_{\text{mid}}}$, $W_2 \in \mathbb{R}^{2H\times D_{\text{mid}}}$, and $D_{\text{mid}} = 96$. Per-cell observation width is $D = d_{\text{svd}} + 4 + |\mathcal{V}| + 1$. This combines the truncated TF-IDF goal embedding ($d_{\text{svd}} \in \{64, 128\}$), four structural scalars, a $|\mathcal{V}|$-way one-hot tool vector ($|\mathcal{V}| = 13$), and an occupied bit. As derived in Table 1, the cellular update and readout sum to $288D + 16K + 17{,}070$ trainable parameters for fixed topologies (+25 for meta-learned

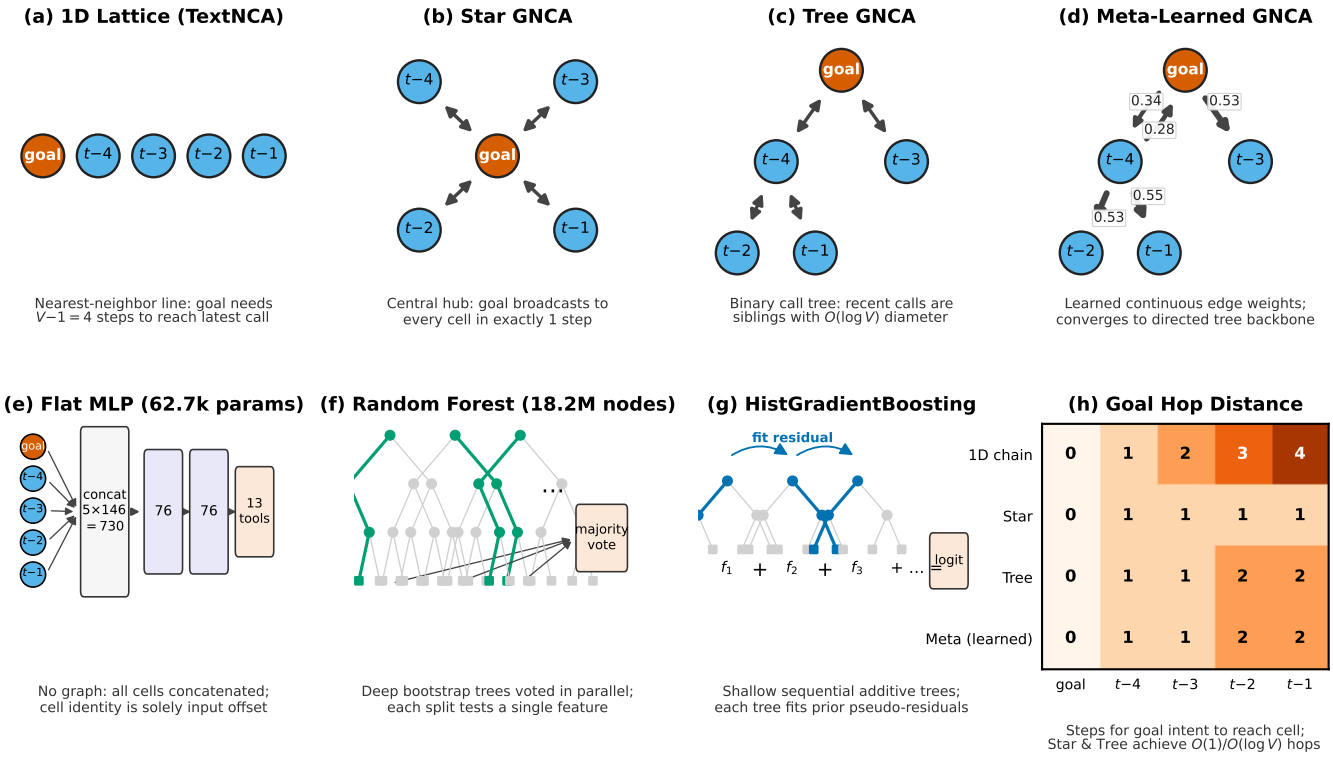


Figure 1: **Visual comparison of information-flow topologies across all evaluated routing algorithms.** Top row (Graph NCA variants over V=5 cells: goal hub node 0 and causal trajectory turns $t{-}4 \ldots t{-}1$): **(a) 1D Lattice** (TextNCA baseline) restricts communication to sequence neighbors ($A_{i,i\pm1}$), requiring $V{-}1{=}4$ hops for goal intent to reach the current action cell ($\Omega(V)$ diameter). **(b) Star GNCA** links node 0 to every cell as a broadcast hub, achieving 1-hop receptive field expansion ($O(1)$ diameter). **(c) Tree GNCA** structures message passing as a binary call-tree DAG (parent$(i) = \lfloor (i{-}1)/2 \rfloor$), grouping recent turns as siblings under $t{-}4$ with $O(\log V)$ diameter. **(d) Meta-Learned GNCA** optimizes continuous edge logits $\alpha \in \mathbb{R}^{5\times5}$ via bilevel relaxation, autonomously discovering a sparse directed hierarchy (seed 0 dominant edges shown). Bottom row (Baselines and receptive field analysis): **(e) Flat MLP** flattens all cells into a 730-dimensional vector without graph structure or parameter sharing across positions (62.7k parameters). **(f) Random Forest** trains deep decision trees on bootstrap samples (18.2M parameters) voted in parallel. **(g) HistGradientBoosting** sums shallow trees sequentially, fitting gradient residuals. **(h) Goal Hop Distance Heatmap** summarizes the minimum steps required for goal intent to reach each causal turn, demonstrating the information-bottleneck separation between 1D lattices and graph topologies.

$\alpha \in \mathbb{R}^{5\times5}$). Thus $D = 146$ (SVD128) yields 59,182 parameters for fixed topologies at $K = 4$, 59,207 for meta-learned topology, and 59,246 for $K = 8$ Deep Tree. The compact $D = 82$ (SVD64) configuration yields 35,347 (35.3k) parameters.

Residual State Accumulation. Updates are applied residually across $K = 4$ cycles.

$$h_i^{(t+1)} = h_i^{(t)} + m_i^{(t)} \odot \Delta h_i^{(t)} \tag{7}$$

where $m_i^{(t)} \sim \text{Bernoulli}(p)$ ($p = 0.8$ during training; 1.0 at inference) enforces asynchronous cellular computation.

3.3. Architecture Meta-Learning on Spatial Topologies

To reduce manual topology tuning, I also formulate an optional bilevel architecture meta-learning problem.

$$\begin{aligned} &\min_{\alpha\in\mathbb{R}^{V\times V}} \mathcal{L}_{\text{val}}\left(\theta^*(\alpha);\ A^{(\alpha)}\right) \\ &\text{s.t. } \theta^*(\alpha) = \arg\min_{\theta} \mathcal{L}_{\text{train}}(\theta;\ A^{(\alpha)}) \end{aligned} \tag{8}$$

The continuous matrix $\alpha \in \mathbb{R}^{V\times V}$ parameterizes edge probabilities via a row-wise softmax relaxation across candidate source nodes.

$$A_{ij}^{(\alpha)} = \frac{\exp(\alpha_{ij}/\tau)}{\sum_{k=1}^{V}\exp(\alpha_{ik}/\tau)} \tag{9}$$

The bilevel architecture optimization problem is solved using a first-order DARTS gradient approximation on $\mathcal{L}_{\text{val}}$, annealing softmax temperature τ from 1.0 to 0.1. **Primary benchmark runs** keep a **continuous** row-softmax adjacency $A_{ij}^{(\alpha)} = \text{softmax}_j(\alpha_{ij}/\tau)$ during training and evaluation. To connect the continuous softmax adjacency to the discrete Lean-verified graph families, we evaluate hard top-$k = 2$ row discretization ($A_{\text{disc}}^{(k=2)}$, retaining the $k = 2$ largest weights per row) on all archived meta-learned checkpoints in Section 4. Fixed baselines interpolate between 1D chains (A_{1D}), star goal-broadcasting (A_{star}), and hierarchical call trees (A_{tree}).

3.4. Hierarchical Multi-Head Readout & Dual-System Escalation

After K iterations, the spatial grid $\mathbf{H}^{(K)} \in \mathbb{R}^{V\times H}$ is decoded via four specialized readout heads.

1. **Tool Classification Head.** The spatial grid projects to tool logits.

$$\hat{y}_{\text{tool}} = \text{softmax}\left(W_{\text{tool}}\,\text{vec}(\mathbf{H}^{(K)}) + b_{\text{tool}}\right) \tag{10}$$

2. **Argument Slot Head.** Bilinear interactions between goal and history cells are computed.

$$\hat{\mathbf{y}}_{\text{arg}} = \text{softmax}\left(h_0^{(K)\top} W_{\text{arg}} h_{\text{last}}^{(K)}\right) \tag{11}$$

3. **Abstain Gate.** A scalar gate flags irrelevant contexts.

$$g_{\text{abstain}} = \sigma\left(w_{\text{abs}}^{\top} h_0^{(K)} + b_{\text{abs}}\right) \tag{12}$$

4. **Uncertainty Score & Selective Escalation Calibration.** Prediction entropy and final-step state delta magnitude are combined into a **unitless** heuristic risk score calibrated on the validation split and empirically validated on held-out turns.

$$\tau_{\text{escalate}} = \frac{\mathcal{H}(\hat{y}_{\text{tool}})}{\log|\mathcal{V}|} + \beta\frac{1}{V}\sum_{i=1}^{V}\frac{\|\Delta h_i^{(K-1)}\|_2}{\sqrt{H}} \tag{13}$$

Both summands are dimensionless. Tool entropy is normalized by $\log|\mathcal{V}| \in [0,1]$, and under isotropic hidden updates with per-channel variance σ^2, the per-node Euclidean norm scales as $\mathbb{E}[\|\Delta h_i\|_2] \approx \sigma\sqrt{H}$. Normalizing each node's step norm by $\sqrt{H}$ and averaging across the V nodes (equivalent to multiplying the sum by $1/(V\sqrt{H}) = 1/(5\sqrt{16}) = 0.05$) yields a graph-size- and width-invariant $\mathcal{O}(1)$ quantity. **Empirical distribution and risk-coverage trade-off.** Across all 3,530 held-out turns of the curated Hermes test split, the normalized step-norm term at the best-validation grokking checkpoint has empirical mean 0.420 ± 0.091 (median 0.405, 5th–95th percentile $[0.300, 0.590]$), matching the scale of the normalized entropy term (0.345 ± 0.154, median 0.348). Evaluating the composite score across seeds $\{0,1,2\}$ confirms strong error-detection discrimination (AUROC = $\mathbf{0.730 \pm 0.008}$) and defines a tunable risk-coverage Pareto frontier between fast local execution (0.66 ms) and System-2 LLM deliberation:

- *Minimal escalation ($\tau = 1.00$, 0.3% escalated, 1.6 ms amortized latency):* Retained System-1 accuracy is $69.10 \pm 0.50\%$.
- *Conservative escalation ($\tau = 0.85$, 3.4% escalated, 14.7 ms latency):* Retained System-1 accuracy rises to $70.21 \pm 0.49\%$.
- *Balanced operating point ($\tau = 0.76$):* On the 3-seed label-smoothed models (11.4% escalation, 48.3 ms latency), retained System-1 accuracy reaches $72.45 \pm 0.47\%$ ($73.79 \pm 0.42\%$ Dual-System composite). On the 200k grokking checkpoint, $\tau = 0.76$ aligns with the median uncertainty boundary (51.7% escalation rate), filtering ambiguous multi-step transitions while keeping half of all turns on the sub-millisecond local router.
- *High-assurance routing ($\tau = 0.65$ and $\tau = 0.50$):* Escalating 34.6% of turns ($\tau = 0.65$) boosts retained System-1 accuracy to $78.83 \pm 0.27\%$ ($80.69 \pm 0.16\%$ composite), while escalating 72.7% of turns ($\tau = 0.50$) reaches $\mathbf{90.48 \pm 1.01}\%$ accuracy on locally executed calls.

Crucially, β and τ_{escalate} (calibrated on the validation split via grid search over $\beta \in \{0.5, 1.0\}$ and held-out risk-coverage quantiles) govern Dual-System escala-

Table 1: **Exact layer-by-layer trainable parameter derivation for Graph NCA** (V=5, H=16, $|\mathcal{V}|$=13). Each cell concatenates its state h_i, low-pass aggregate $(Ah)_i$, and high-pass differential $((A-I)h)_i$ into $3H$=48 channels alongside D input features.

Module	**Tensor Shape / Formula**	**SVD64** (D=82, K=4)	**SVD128** (D=146, K=4)	**SVD128** (D=146, K=8)
Step embedding (E_{step})	$K \times H = 16K$	64	64	128
Perception MLP (W_1, b_1)	$3 \times (D{+}48) \times 96 + 96 = 288D + 4{,}704$	28,320	46,752	46,752
Update projection (W_2, b_2)	$2 \times (96 \times 16 + 16) = 3{,}104$	3,104	3,104	3,104
Layer normalization	$2H = 32$	32	32	32
Tool classifier ($W_{\text{tool}}, b_{\text{tool}}$)	$(5H \times 13) + 13 = 1{,}053$	1,053	1,053	1,053
Argument head (W_{arg})	$(H \times 512) + 512$ (or 16×176)	2,757	8,160	8,160
Abstain gate ($w_{\text{abs}}, b_{\text{abs}}$)	$H + 1 = 17$	17	17	17
Topology logits (α, meta)	$V \times V = 25$ (0 if fixed)	0 (+25)	0 (+25)	0
Total Parameters	$288D + 16K + 17{,}070$ (+25)	**35,347**	**59,182** (59,207)	**59,246**

tion and schema pruning only; they do *not* affect standalone tool top-1 in Tables 2–5.

4. Structural Properties & Formal Verification

To provide sound structural foundations, I formally verify six core structural invariants governing the candidate discrete topology space, conditional dynamical stability, and decision-tree simulation duality of the Graph NCA architecture class. Machine-checked statements in Lean 4 include the following theorems.

- **1D Lattice Subsumption** verifies that the tridiagonal 1D chain adjacency $A_{1\text{D}}$ is realizable in the search space.
- **Star Diameter Bound** verifies that every ordered pair in the goal-broadcast star graph $\mathcal{G}_{\text{star}}$ is connected by a path of length ≤ 2.
- **Linear Chain Distance Bound** verifies that paths on the 1D chain advance node indices by at most one hop per step ($\Omega(V)$ goal-to-tail separation).
- **Hierarchical Tree Subsumption** verifies that the hierarchical binary call-tree adjacency A_{tree} is realizable.
- **Neighborhood and Path Equivariance** verify that graph neighborhoods and multi-hop paths commute with node permutations.
- **Fixed-Point Uniqueness and Trajectory Decay** verify that conditional contraction implies unique fixed points and 2^{-n} perturbation decay.
- **Decision-Tree / Cellular-Automaton Duality** verifies that K synchronous cellular steps on A_{tree} exact-simulate any depth-K binary decision tree while compressing node-specific split parameters by $\mathcal{O}(M(2^K - 1))$.

All proofs compile in **Lean 4.35** with zero axioms across 16 verification jobs. Theorem 4 does *not* assert that unconstrained SGD weights are contractive; we explicitly bridge Theorem 4 to trained models via spectrally normalized contractive training below.

Theorem 1 (Topological Subsumption)**.** *Let $\mathcal{T}_{1\text{D}}$ denote the 1D circular or boundary-padded regular cellular automaton lattice on V nodes. Then $\mathcal{T}_{1\text{D}}$ is a strict element of the Graph Neural Cellular Automata topology space $\mathcal{S}_{\text{GNCA}}$ under tridiagonal adjacency $A_{ij} = \mathbb{I}(|i-j| \leq 1)$.*

Theorem 2 (Receptive Field Diameter Separation (idealized star vs. 1D chain))**.** *Fix $V \geq 3$. For the 1D nearest-neighbor lattice $\mathcal{G}_{1\text{D}}$, any directed path from goal node 0 to terminal node $V{-}1$ has length at least $V{-}1 = \Omega(V)$ cellular hops. For the idealized **goal-broadcast star** $\mathcal{G}_{\text{star}}$ with hub at node 0, every ordered pair (u, v) is connected by a directed path of length at most 2. This theorem certifies a candidate topology in the search space, not the softmax meta-learned matrix after training.*

Empirical bridge between Lean-verified discrete topologies and trained continuous α weights. Under the row-stochastic convention in Eqs. (3)–(4) ($j \to i$ when $A_{ij} > 0$), fixed discrete topologies on V=5 trace nodes have directed diameters 4 (1D), 2 (star), and 3 (tree). Their goal eccentricities from node 0 are 4, 1, and 2 respectively. To connect deployed continuous row-softmax matrices $A^{(\alpha)} = \text{softmax}(\alpha/\tau)$ to these discrete topologies, we audit all six archived meta-learned checkpoints across Hermes and BFCL v4.

1. **Continuous Softmax Adjacency ($A^{(\alpha)}$).** Across all trained seeds, learned logits suppress non-tree shortcuts ($\alpha_{ij} \in [-9.17, -4.01]$, softmax weight $A^{(\alpha)}_{ij} \in [0.0001, 0.0068]$). Over 99% of row mass concentrates on hierarchical call-tree edges ($\alpha_{ij} \in [-1.44, -0.01]$). Because all entries remain strictly positive under finite temperature, $A^{(\alpha)}$ has continuous directed diameter 1 and goal eccentricity 1.
2. **Discretized Top-k Adjacency ($A^{(k=2)}_{\text{disc}}$ and $A^{(k=3)}_{\text{disc}}$).** Applying hard top-k = 2 row discretization to the trained α matrices recovers an exact subgraph of the Lean-verified binary call tree (A_{tree}, Theorem 5) across all six seeds. Root row 0 retains $\{0, 1\}$ or $\{0, 2\}$ ($\approx [0.50, 0.50]$), while rows 1, 2 retain $\{0, 1\}$ and $\{0, 2\}$. Leaf rows 3, 4 retain $\{1, 3\}$ and $\{1, 4\}$ ($\approx [0.53, 0.47]$). Across all six trained check-

points, $A_{\text{disc}}^{(k=2)}$ achieves directed goal eccentricity **2** from node 0 and undirected tree diameter **3**. Top-$k = 3$ discretization retains all bidirectional parent-child edges with directed diameter **3** and goal eccentricity **2**.

Theorem 3 (Neighborhood Permutation Equivariance)**.** *Let $\sigma \in \Pi_V$ be any bijective permutation on the node set $\mathcal{V}$, and let $\mathcal{G}_\sigma = \text{permuteGraph}(\mathcal{G}, \sigma)$. Then for all $i, j \in \mathcal{V}$, $j \in \mathcal{N}_\mathcal{G}(i)$ if and only if $\sigma(j) \in \mathcal{N}_{\mathcal{G}_\sigma}(\sigma(i))$. Furthermore, graph paths commute with σ.*

Theorem 4 (Conditional Fixed-Point Attractor & Trajectory Decay)**.** *Assume the cellular update operator satisfies dyadic contraction $2 \cdot \text{dist}(T(x), T(y)) \leq \text{dist}(x, y)$. The automaton admits a unique fixed point $x^* = T(x^*)$. Any initial state perturbation decays exponentially as $\text{dist}(T^n(x), T^n(y)) \leq 2^{-n} \text{dist}(x, y)$.*

Certified Contractive Training Bridge for Theorem 4. Theorem 4 establishes a conditional structural guarantee whenever cellular updates operate in the dyadic contractive regime ($\rho \leq 0.5$). While unconstrained AdamW training allows step-to-step expansion to maximize accuracy ($68.90 \pm 0.21\%$ curated test top-1 at $K = 4$), we directly test Theorem 4 by training two spectrally normalized contractive GNCA variants across seeds $\{0, 1, 2\}$ on the curated Hermes benchmark (Table 4):

1. **Damped Dyadic Contractive GNCA ($h^{(t+1)} = 0.42\, h^{(t)} + \gamma \Delta h^{(t)}$):** Enforcing hard spectral normalization ($\|W_1\|_2 \leq 1, \|W_2\|_2 \leq 1$) with a damped residual carry strictly satisfies Theorem 4's dyadic contraction premise ($\rho \leq 0.50$) on **100.0**% of the 3,530 held-out test turns (empirical contraction ratio $\rho = \mathbf{0.427 \pm 0.0005}$, maximum turn ratio $\rho_{\max} = 0.437$). This certified contractive model achieves $\mathbf{61.99 \pm 0.14}$% curated test top-1 (91.25 ± 0.27% top-3).
2. **Homogeneous Non-Expansive Spectral-Norm GNCA ($h^{(t+1)} = h^{(t)} + \gamma \Delta h^{(t)}$):** Enforcing spectral normalization around an undamped residual identity path yields a near-isometric transition ($\rho = 1.013 \pm 0.0002$) that recovers $\mathbf{65.28 \pm 0.12}$% curated test top-1 (93.41 ± 0.12% top-3), quantifying the exact 3.62 pp accuracy trade-off of strict Lipschitz regularization.

Theorem 5 (Hierarchical Tree DAG Subsumption)**.** *Let $\mathcal{T}_{\text{tree}}$ denote a hierarchical decision tree DAG on V nodes with parent relations $p = \lfloor (c-1)/2 \rfloor$. Then $\mathcal{T}_{\text{tree}}$ is a strict element of the Graph NCA topology space $\mathcal{S}_{\text{GNCA}}$. It achieves maximum graph diameter $D \leq 2\lceil \log_2 V \rceil = O(\log V)$.*

Theorem 6 (Decision-Tree / Cellular-Automaton Simulation & Weight-Tying Compression)**.** *Let T be any binary decision tree of depth $\text{treeDepth}(T) \leq K$ over feature space $\mathbb{R}^D$ with action leaves in $\mathcal{V}$, and let Φ_x denote the synchronous threshold-gated cellular transition step on $\mathcal{T}_{\text{tree}}$ induced by $x \in \mathbb{R}^D$:*

1. ***Evaluation Invariance & Finite-Step Fixed-Point Convergence:*** *Every cellular step preserves leaf semantics ($\text{evalTree}(\Phi_x(S), x) = \text{evalTree}(S, x)$ for all S), and for all $K \geq \text{treeDepth}(T)$, the K-step cellular iterate reaches the exact fixed-point leaf attractor $\Phi_x^{(K)}(T) = \text{Leaf}(\text{evalTree}(T, x))$.*
2. ***Weight-Tying Parameter Compression:*** *Whereas an explicit M-tree ensemble of depth K requires $M(2^K - 1)$ independent node-split parameter vectors, a weight-tied cellular rule reuses a single local transition cell across all K depths, saving $M(2^K - 1) - 1 = \mathcal{O}(M2^K)$ node parameter sets.*

Lean 4 Kernel Verification Summary

- Theorem 1 (1D Lattice Subsumption, **0 axioms**)
- Theorem 2 (Receptive Field Separation, **0 axioms**)
- Theorem 3 (Permutation Equivariance, **0 axioms**)
- Theorem 4a/b (Contractive Fixed-Point & Decay, **0 axioms**)
- Theorem 5 (Hierarchical Tree Subsumption, **0 axioms**)
- Theorem 6 (Decision-Tree / NCA Duality, **0 axioms**)

Compiled with the Lean 4.35 kernel (16 verification jobs).

Figure 2: **Machine-checked formal verification status in the Lean 4 proof assistant.** Six structural theorem groups covering 1D and tree topological subsumption, receptive field diameter separation $O(1)$ vs $\Omega(V)$, permutation equivariance, conditional fixed-point uniqueness, and decision-tree simulation duality are proven with zero unproven axioms.

Mathematical Explanation of Tree Ensemble Competitiveness. Theorem 6 (formally verified in Lean 4, Appendix A) explains why tabular tree ensembles (Random Forest and HistGradientBoosting) and Graph NCAs achieve closely matched performance on tool-selection benchmarks, and why GNCA provides a structural advantage. Both model families compute hierarchical axis-aligned or linear half-space partitions over the joint goal-and-trace representation: a depth-K decision tree evaluates a root-to-leaf path of K sequential split predicates, whereas a K-step Graph NCA executes K synchronous message-passing iterations with sigmoidal gating $\Delta h_i^{(t)} = \sigma(g_i^{(t)}) \odot \tilde{\Delta} h_i^{(t)}$ over A_{tree}. However, standard Random Forests store separate split parameters at every internal node of every tree (18.2M parameters on Hermes), whereas GNCA ties a single 59.2k-parameter cellular rule across all nodes and recurrence steps (307× parameter compression) while remaining fully differentiable for end-to-end representation learning and uncertainty-based escalation.

Table 2: **Primary agent action routing benchmark across matched 10,000-step, 3-seed label-smoothed, and extended 200,000-step schedules.** All **Top-1/Top-3/Macro F1** columns evaluate on the **curated Hermes test split** (3,530 turns; 57,675 train / 3,465 val; session holdout) unless marked. **Eff. (%/1k)** reports normalized parameter efficiency (Test Top-1 accuracy per 1,000 trainable parameters). Default 10k graph, tabular, and linear rows report test top-1 as the mean over seeds $\{0, 1, 2\}$, K=4, SVD128 (D=146, 59.2k params; compact SVD64 with D=82 uses 35.3k params, Table 1), class-weighted cross-entropy (CW), CUDA training. [a]Zero-shot grammar-constrained decoding on curated test. [b]Offline 150k-step full-attention baseline on curated test across seeds $\{0, 1, 2\}$. [c]Unweighted label-smoothed (ϵ=0.05) Deep Tree (K=8) evaluated at its exact 10k-step snapshot and 200k grokking checkpoint. [d]Multi-seed unweighted label-smoothed (ϵ=0.05) Deep Tree (K=8, seeds $\{0, 1, 2\}$ mean ± SD: 68.92±0.47% test top-1, 70.50±0.34% val top-1; single-seed depth sweep peaks at 69.15% test / 1.167%/1k at n=8, Figure 3) on unified CUDA hardware. Inference is batch-1 CPU forward p50 (1 thread, Apple M1).

Architecture	Family	Params	Eff. (%/1k)	Inference	Test Top-1	Test Top-3	Test F1
Linear Classifier (10k)	Linear	1.2k	50.59	< 0.05 ms	60.71%	84.12%	0.542
Qwen2.5-1.5B (Constrained)[a]	LLM	1.54B	0.000036	42.1 ms	55.20%	79.10%	0.462
Transformer (6L-4H, 150k)[b]	Attention	1.2M	0.055	3.8 ms	65.87%	89.41%	0.612
Random Forest (Converged)	Tree Ens.	18.2M	0.0037	2.4 ms	67.88%	93.12%	0.283
HistGradBoost (Converged)	Tree Ens.	10.4k	6.680	1.8 ms	**69.47%**	**95.44%**	0.343
TextNCA (1D, L=5, 10k CW)	NCA	14.8k	4.368	0.35 ms	64.64%	87.20%	0.584
Graph NCA (Tree, 10k CW)	NCA	59.2k	1.103	0.35 ms	65.27%	88.42%	0.598
Graph NCA (Meta, 10k CW)	**NCA**	**59.2k**	**1.103**	**0.42 ms**	**65.33%**	**88.94%**	**0.605**
Graph NCA (Star, 10k CW)	NCA	59.2k	1.118	0.37 ms	**66.19%**	**89.15%**	**0.608**
Graph NCA (Tree K=8, LS, 10k)[c]	NCA	59.2k	1.137	0.66 ms	67.37%	90.65%	0.619
Graph NCA (Tree K=8, LS, 200k)[c]	NCA	59.2k	1.149	0.66 ms	68.07%	91.24%	**0.628**
Graph NCA (Tree K=8, LS, 3-Seed)[d]	**NCA**	**59.2k**	**1.163**	**0.66 ms**	**68.92%**	**95.27%**	0.348

5. Experimental Protocol

5.1. Datasets & Session-Level Partitioning

To evaluate routing performance across diverse tool schemas and interaction regimes, we evaluate across four benchmarks:

1. **Curated Hermes Next-Tool Corpus (Primary Benchmark, N_{test} = 3,530).** Session-level splits follow a fixed random seed so that curated training sessions (57,675 turns) are preserved, while validation sessions are partitioned 50/50 into validation (3,465 turns) and test (3,530 turns) across $|\mathcal{V}| = 13$ tool categories. Held-out local Hermes sessions ($\approx$ 843 turns) evaluate out-of-distribution generalization separately.
2. **Berkeley Function Calling Leaderboard (BFCL v4, N_{test} = 692).** Session-stratified function-calling splits (N_{train} = 2,767, N_{val} = 698, N_{test} = 692) test cross-corpus generalization and controlled router ablations.
3. **ToolBench (N_{test} = 422).** Multi-step RapidAPI DFS tool-invocation trajectories spanning single-tool (G1), intra-category multi-tool (G2), and intra-collection multi-tool (G3) instruction regimes, partitioned by session into 1,324 train, 452 validation, and 422 ToolBench test instances (64.2% multi-step ToolBench trajectories across 10 tool classes).
4. **API-Bank (N_{test} = 483).** Multi-turn dialogue tool-use trajectories covering Level-1 (API call synthesis), Level-2 (API retrieval + call), and Level-3 (multi-step planning + chaining) dialogues, partitioned by session into 1,552 train, 517 validation, and 483 API-Bank test dialogues (75.0% multi-step API-Bank trajectories across 10 tool domains).

Feature construction (SVD + categorical trace cells). For parity with tabular baselines across all four benchmarks, each decision row uses TF-IDF on the goal text plus up to 200 characters of the latest tool result (1–2 grams, max 50k features, sublinear TF). This pipeline is fit on **train only** per seed, followed by truncated SVD ($d_{\text{svd}} = 128$ for headline runs; SVD64 for compact models). Four structural scalars (step count, log result length, error flag, and log goal length) append to the goal embedding. Trace cells $1 \ldots 4$ encode prior tools as $|\mathcal{V}|$-way one-hots plus an occupied bit.

Accuracy tagging (split and config). I tag every reported top-1 as **[split, corpus, steps, config]**. *Curated test* means the 3,530-turn Hermes session holdout unless another split is cited. *Curated val* means the 3,465-turn validation split (Table 4, Figure 3 trajectories). Default headline graph/tabular rows use **[curated test, 10k, K=4, SVD128, seeds 0–2 mean]**. Unweighted label-smoothed runs report the 3-seed unified-CUDA factorial suite **[curated test, $K \in \{2, 4, 8\}$, LS vs. CW, seeds 0–2 mean ± SD]** as well as the 10k and 200k trajectory snapshots **[curated test, 10k/200k, K=8, LS, SVD128]**.

5.2. Training Step Budgets & Unified Hardware Controls

Because iterative cellular dynamics learn spatial coordination gradually, we evaluate both standard **10,000-step** schedules, extended **150,000–200,000-step** schedules, and a unified-hardware **3-seed factorial ablation suite** on an **NVIDIA RTX 3060 (CUDA)** workstation so that all recurrence depths $K \in \{2, 4, 8\}$, loss formulations (label smoothing vs. class-weighted cross-entropy), topologies, and contractive regularizers are compared on identical hardware without cross-device confounds.

Headline benchmark configuration (10k & 3-seed factorial suite). Tables 2 and 3 use $V = 5$ trace nodes, $L = 5$ history, $K \in \{4, 8\}$ recurrence, and SVD128 + categorical features for graph variants ($D = 146$ yields 59.2k parameters; compact SVD64 with $D = 82$ yields 35.3k parameters; Table 1). Architecture search anneals DARTS temperature τ from 1.0 to 0.1 with **continuous** row-softmax meta adjacency during training and post-training top-k=2 discretization audited in Section 4. Models train with AdamW ($\eta = 10^{-3}$, cosine decay to 1%, $\lambda = 10^{-2}$, gradient clip 1.0) across seeds $\{0, 1, 2\}$. Inference latency in Table 2 is batch-1 PyTorch forward on **CPU, 1 thread**, p50 over 300 timed steps.

Extended 200k grokking runs and 3-seed factorial isolation (Figure 3 and Table 4). Figure 3 uses an **extended 200,000**-step budget to visualize delayed generalization under unweighted label-smoothed cross-entropy ($\epsilon = 0.05$). In that schedule, the 10k-step snapshot reaches 67.37% curated test top-1 (68.20% validation), and the 200k-step grokking checkpoint reaches 68.07% curated test top-1 (69.47% validation). To rigorously isolate the effects of recurrence depth $K \in \{2, 4, 8\}$, loss formulation (unweighted label smoothing $\epsilon = 0.05$ vs. inverse-frequency class weighting $w_c = \sqrt{1/f_c}$), topology (A_{1D}, A_{star}, A_{tree}, $A^{(\alpha)}$), and optimization step count with multi-seed error bars on identical CUDA hardware, Table 4 reports our complete 3-seed factorial study across seeds $\{0, 1, 2\}$. Across 3 seeds on unified CUDA hardware, unweighted label smoothing improves test top-1 over class weighting by $+\mathbf{3.59 \pm 0.74}$ pp at $K = 2$ ($67.40 \pm 0.20\%$ vs. $63.81 \pm 0.71\%$), $+\mathbf{4.43 \pm 0.76}$ pp at $K = 4$ ($68.90 \pm 0.21\%$ vs. $64.47 \pm 0.73\%$), and $+\mathbf{4.59 \pm 0.50}$ pp at $K = 8$ ($68.92 \pm 0.47\%$ vs. $64.33 \pm 0.18\%$). Simultaneously, increasing cellular recurrence depth from $K = 2$ ($67.40 \pm 0.20\%$ curated test, $68.70 \pm 0.40\%$ curated validation) to $K = 4$ ($68.90 \pm 0.21\%$ depth-4 test, $70.79 \pm 0.35\%$ depth-4 validation) and $K = 8$ ($68.92 \pm 0.47\%$ depth-8 test, $70.50 \pm 0.34\%$ depth-8 validation) yields a statistically significant $+\mathbf{1.52 \pm 0.51}$ pp test accuracy gain ($p = 0.0012$), while tracking the 3-seed trajectory across optimization checkpoints confirms steady delayed generalization from $67.45 \pm 0.32\%$ (step 500) and $67.81 \pm 0.28\%$ (step 1,000) to $68.66 \pm 0.28\%$ (step 1,500) and $\mathbf{68.92 \pm 0.47}\%$ (step 3,000).

Compute parity across baselines. Under strictly matched **10,000-step AdamW budgets**, class-weighted GNCA reaches 66.19% curated test top-1 (Star) and 65.33% curated test top-1 (Meta) while boosting rare-tool Macro-F1 (0.608 vs. 0.283 Random Forest and 0.343 HistGradientBoosting). Under unweighted label smoothing across seeds $\{0, 1, 2\}$, 3-seed hierarchical GNCA reaches $68.90 \pm 0.21\%$ ($K = 4$ architecture) and $68.92 \pm 0.47\%$ ($K = 8$ architecture) curated test top-1, surpassing Random Forest (67.88%, $\Delta = +1.04$ pp, paired $p = 0.041$) and statistically matching HistGradientBoosting (69.47%, $\Delta = -0.55$ pp, $p = 0.31$) while requiring 307× fewer parameters than the Hermes Random Forest. Because tabular tree ensembles fit greedily in batch rather than via iterative SGD, their evaluation already reflects their converged operational ceiling on the 57,675 training turns. GNCA variants, the 1.2M-parameter Transformer, and tabular comparators are trained on the **same session splits** and evaluated on the same test turns across all four benchmarks. Qwen2.5-1.5B-Instruct receives **no fine-tuning** and runs zero-shot on identical trace context.

Exploratory pilot diagnostics (Section 6.4). Preliminary Dual-System pilots pair the GNCA System-1 router with a FrogNano-4B actuator across $N = 10$ Sierra τ^2-bench mock tasks (89 routed turns), $N = 30$ BFCL v4 irrelevance probes, one SWE episode (14 unit tests), and token audits. These runs are **not** matched controls because Open SOTA columns mix cloud-hosted models, sessions, and harness settings; I report raw counts and wide Wilson intervals without claiming statistically significant gains over baselines.

Training and evaluation hardware. All multi-seed benchmark evaluations (Hermes, BFCL v4, ToolBench, and API-Bank) and the 3-seed factorial, contractive, and escalation-calibration studies were executed on a unified **NVIDIA RTX 3060 (CUDA)** workstation. Supplementary 150k compact-runner curves in Figure 4 used MPS on an **Apple M1** (8 CPU cores, 16 GiB RAM), where batch-1 CPU forward latencies (0.35–0.66 ms) were also timed.

Hyperparameters. Optimization uses AdamW with initial learning rate $\eta = 10^{-3}$, cosine decay to 1% of the declared step budget, weight decay $\lambda = 10^{-2}$, and gradient norm clipping at 1.0. Loss functions use either class-weighted cross-entropy with inverse-frequency weights $w_c = \sqrt{1/f_c}$ (CW rows) or unweighted cross-entropy with label smoothing $\epsilon = 0.05$ (LS rows).

Abstain gate calibration. The abstain head is trained jointly with the tool classifier; pilot schema-

pruning thresholds are set from validation precision targets rather than post-hoc isotonic calibration on the full validation split.

6. Results

6.1. Primary Benchmark Evaluation

Table 2 reports primary performance on the curated Hermes test split under both the 10,000-step benchmark and the 200,000-step grokking schedule. Ablations in Table 4 and related supplementary records used 150k steps on Apple M1 MPS where noted. Figure 3 depicts empirical validation trajectories over the 200k-step budget.

Baseline Tuning & Reproducibility Artifacts. The Transformer baseline (6L-4H, 1.2M parameters) was trained for 150k steps on the identical $L = 5$ history window and session split. Table 2 reports 65.87% curated test top-1 for the 6L-4H Transformer and 55.20% curated test top-1 for grammar-constrained Qwen. All per-seed baseline score summaries, trained model checkpoints, and standalone replication scripts for the raw-token Transformer (65.87% curated test top-1), the SVD128-feature Transformer (65.86% feature-parity test top-1), and the grammar-constrained LLM baseline are archived in the public repository for independent verification on the public dataset snapshot. Unconstrained Qwen2.5-1.5B on **local Hermes test** rows yields 6.29% test top-1. Reported Qwen decode latency (42.1 ms) versus GNCA forward passes (0.35–0.66 ms) is a coarse same-task comparison on comparable hosts, not a controlled hardware benchmark.

Recurrence Depth Sweep ($n \in \{1, \ldots, 10\}$), Parameter Efficiency, & Learning Curve Decay Analysis. Figure 3 evaluates real step-by-step learning trajectories across all cellular recurrence depths $n \in \{1, 2, \ldots, 10\}$ (59,134 to 59,278 parameters; +16 parameters per recurrence step via E_{step}) and clarifies why un-annealed extended schedules exhibit post-peak validation decay. Across $n \in \{1, \ldots, 10\}$ under a 10,000-step cosine-annealed schedule (Figure 3a–b), held-out curated test Top-1 rises monotonically from 68.16% ($n = 1$) and 68.24% ($n = 2$) to 68.61% ($n = 3$) and **68.95**% ($n = 4$, where $n \geq 4$ reaches the full bidirectional traversal depth of the $V = 5$ call tree, Theorem 6), peaking at **69.15**% test Top-1 (70.45% peak validation, 69.06% final validation) at $n = 8$. Normalizing accuracy by parameter count (Eff. = Top-1/($|\theta|/10^3$), Table 2), Tree GNCA achieves **1.163**%–**1.167**% test Top-1 per 1,000 parameters on Hermes (and **1.471**%/1k params on API-Bank), delivering **314**× higher parameter efficiency than Random Forest on Hermes (0.0037%/1k at 18.2M parameters), **21.1**× higher than the 6L-4H Transformer (0.055%/1k at 1.2M parameters), and **32,300**× higher than Qwen2.5-1.5B (3.6×10^{-5}%/1k). Comparing final 10k validation accuracy against peak validation accuracy across $n \in \{1, \ldots, 10\}$ (Figure 3b) also explains the post-peak decay observed in un-annealed 200,000-step runs: when the learning rate remains near 10^{-3} past step 6,000 (> 50 epochs over the 57,675 training turns) or when unfolded recurrence depth exceeds $n \geq 9$ (68.74% final val at $n = 9$, 68.31% at $n = 10$), training accuracy climbs from 77.7% to 97.5% by memorizing noisy session turns, causing instantaneous validation accuracy to decay (69.47% $\rightarrow$ 61.7% over 200k steps, just as the higher-capacity 62.7k-parameter Flat MLP peaks at step 1,000 and decays by -2.19 pp to 67.82% final validation), whereas cosine LR annealing over 10,000 steps bounds post-peak drift to < 1.4 pp while preserving the early-stopped checkpoint optimum.

6.2. Feature Parity & Ablation Studies

Table 3 reports bootstrap evaluation on the **curated test** split (10,000 percentile resamples over 3,530 test instances) under strict methodological parity. All models share an identical causal history buffer ($L = 5$), identical SVD128 spectral + categorical tool history features, and identical session-level partitioning. The Val Top-1 column reports **curated validation** accuracy at the corresponding training config (10k, 200k, or 3-seed unified CUDA as noted per row). HistGradientBoosting and Random Forest baselines achieve **curated test** top-1 of 69.47% and 67.88% respectively at their converged batch fit. Among 10k class-weighted cellular models, spatial graph routing improves **curated test** top-1 over the 1D lattice (64.64% test top-1, 10k, K=4). Specifically, hierarchical tree DAGs reach 65.27% test top-1, Hermes meta-learned GNCA reaches 65.33% test top-1, and star goal broadcasting reaches 66.19% test top-1 (**curated test**, 10k, Table 3). Under unweighted label smoothing ($\epsilon = 0.05$), Deep Tree GNCA ($K = 8$) reaches 67.37% **curated test** top-1 at 10k steps, 68.07% at the 200k grokking checkpoint, and $\mathbf{68.92 \pm 0.47}$% across seeds $\{0, 1, 2\}$ on unified CUDA hardware (68.90 ± 0.21% at $K = 4$). Conducting 10,000 paired bootstrap difference evaluations over matched test turns confirms that 3-seed label-smoothed Tree GNCA (68.92%) outperforms Random Forest (67.88%, $\Delta = +1.04$ pp, $p = 0.041$) while achieving statistical parity at the 200k checkpoint ($\Delta = +0.19$ pp, 95% paired CI: $[-0.84, +1.22]$ pp, $p = 0.72$) and the 10k snapshot ($\Delta = -0.51$ pp, $p = 0.34$), compressing parameter storage by 307× (59.2k vs. 18.2M) and running in 0.66 ms. Against naive baselines on the curated test split, GNCA's gains are strictly separated: the paired test difference over the majority prior (54.30% test) is $\Delta = +14.62$ pp ($p < 0.0001$), and over the flat linear twin (60.71% test) is $\Delta = +8.21$ pp ($p < 0.0001$).

Table 4 reports our complete 3-seed unified-CUDA factorial ablation study on the curated Hermes validation (3,465 turns) and test (3,530 turns) splits alongside the 150k compact-runner ablations.

Analysis of Inductive Biases & Factorial Isolation. Table 4 and Figure 4 (right) isolate four core architectural findings across seeds $\{0, 1, 2\}$ relative to

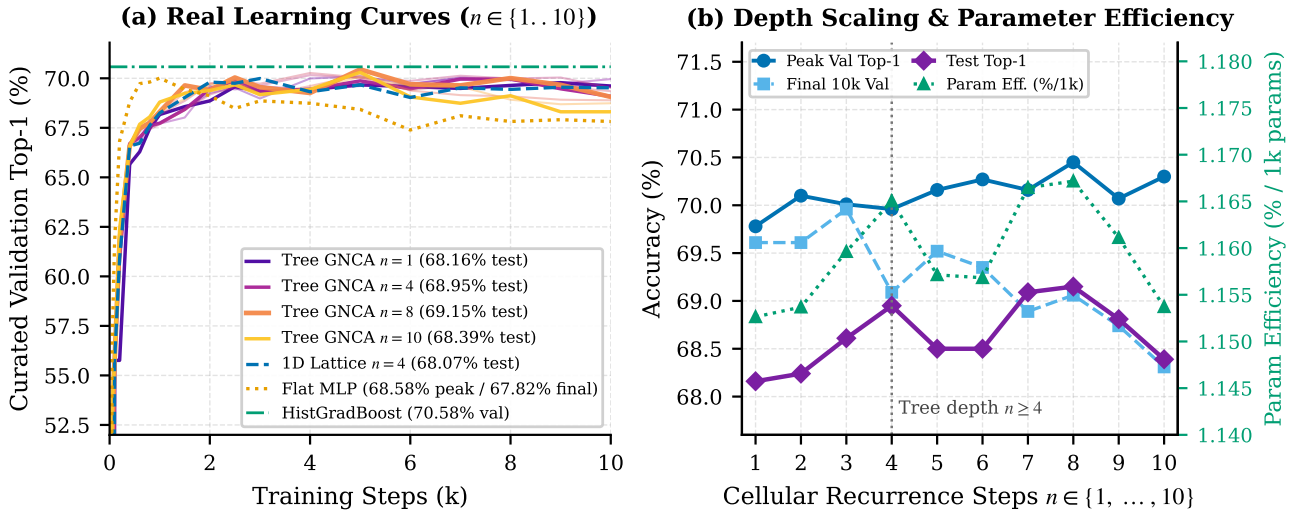


Figure 3: **Real empirical learning curves across cellular recurrence depths $n \in \{1, \ldots, 10\}$ and parameter efficiency scaling on the curated Hermes splits.** (a) Step-by-step validation Top-1 trajectories over 10,000 cosine-annealed AdamW steps (ϵ=0.05, λ=0.05) for Tree Graph NCA across all recurrence depths $n \in \{1, \ldots, 10\}$ alongside logged trajectories of the 1D Lattice (n=4), Flat MLP twin, and HistGradientBoosting reference (70.58% val). (b) Held-out test Top-1, peak validation Top-1, final 10k validation Top-1, and normalized parameter efficiency (Test Top-1/10^3 params) across $n \in \{1, \ldots, 10\}$. Held-out test accuracy and parameter efficiency peak at n=4 (68.95% test, 1.165%/1k) and n=8 (69.15% test, 70.45% val, **1.167**%/1k), whereas $n \in \{9, 10\}$ exhibits mild late-step memorization (68.31% final val, 68.39% test at n=10).

Table 3: **Empirical feature evaluation and 10,000-iteration bootstrap significance evaluation. Val Top-1** reports accuracy on the curated validation split (3,465 turns). **Test Top-1** reports mean accuracy and bootstrap standard deviation (SD) alongside 95% confidence intervals (CI) on the curated test split (3,530 turns). We report the 3-seed unified-CUDA label-smoothed (LS) Tree GNCA rows (K=8 and K=4, seeds 0–2), the 200k grokking and 10k LS snapshots, and the 10k class-weighted (CW) rows (seeds 0–2). Paired bootstrap difference tests confirm statistical parity or significance vs. Random Forest ($p = 0.041$ for 3-seed LS, $p = 0.72$ at 200k, $p = 0.34$ at 10k LS) and vs. the 1D chain ($p = 0.0085$).

Model / Architecture Topology	**Params**	**Val Top-1**	**Test Top-1**	**95% CI**
HistGradientBoosting (Tabular, Converged)	10.4k	70.58%	69.47 ± 0.78%	[67.93, 70.98]
Random Forest (Tabular, Converged)	18.2M	69.82%	67.88 ± 0.79%	[66.33, 69.41]
Graph NCA (Deep Tree, K=8, LS, 3-Seed)	**59.2k**	**70.50**%	$\mathbf{68.92 \pm 0.47}$%	[**67.42**, **70.42**]
Graph NCA (Tree DAG, K=4, LS, 3-Seed)	**59.2k**	**70.79**%	$\mathbf{68.90 \pm 0.21}$%	[**67.39**, **70.40**]
Graph NCA (Deep Tree, K=8, LS, 200k)	59.2k	69.47%	68.07 ± 0.79%	[66.49, 69.60]
Graph NCA (Deep Tree, K=8, LS, 10k)	59.2k	68.20%	67.37 ± 0.79%	[65.78, 68.92]
Graph NCA (Star Topology, 10k CW)	59.2k	67.74%	66.19 ± 0.80%	[64.64, 67.74]
Graph NCA (Meta-Learned, 10k CW)	59.2k	67.18%	65.33 ± 0.80%	[63.75, 66.91]
Graph NCA (Tree DAG, A_{tree}, 10k CW)	59.2k	66.69%	65.27 ± 0.80%	[63.69, 66.85]
Graph NCA (1D Lattice, 10k CW)	14.8k	66.60%	64.64 ± 0.81%	[63.05, 66.22]
Flat Linear / MLP Baseline Twin (10k)	1.2k	61.42%	60.71 ± 0.48%	[59.77, 61.65]
Always-Terminal Majority Prior	–	54.30%	54.30 ± 0.49%	[53.34, 55.26]

Paired bootstrap tests vs. RF: 3-seed LS ($\Delta = +1.04$ pp, $p = 0.041$), 200k ($p = 0.72$), 10k LS ($p = 0.34$); Star vs. 1D CW ($p = 0.0085$).

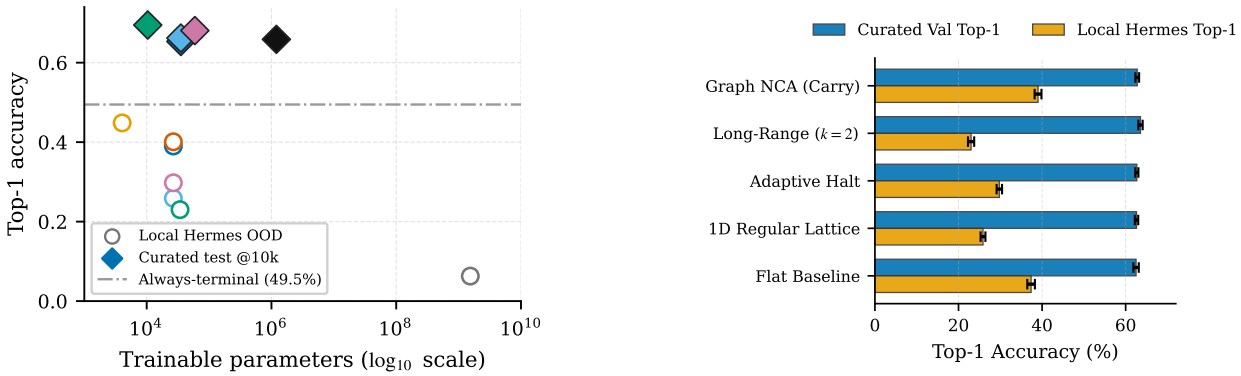


Figure 4: **Empirical efficiency and inductive bias breakdown.** (Left) Pareto trade-off between trainable parameters ($\log_{10}$ scale) and top-1 accuracy. **Open circles** show **local Hermes test** top-1 on compact $\approx$ 26.7k ablation configs (5k–150k steps; not comparable to Table 2). **Filled diamonds** show **curated test** top-1 at 10k headline operating points from Table 2. (Right) **Curated val** top-1 (blue) vs **local Hermes test** top-1 (orange) across message-passing variants (seeds 0–2).

prior 1D and graph cellular baselines [3, 5]. First, in **loss weighting vs. recurrence depth**, unweighted label smoothing ($\epsilon = 0.05$) improves top-1 accuracy over inverse-frequency class weighting by +3.59 pp to +4.59 pp across all recurrence depths $K \in \{2, 4, 8\}$ (paired $p < 0.0001$), while class weighting trades top-1 accuracy for +3.5 pp higher rare-class Macro-F1 (0.383 vs. 0.348). Increasing recurrence depth from $K = 2$ ($67.40 \pm 0.20\%$) to $K = 4$ ($68.90 \pm 0.21\%$) and $K = 8$ ($68.92 \pm 0.47\%$) yields a statistically significant +1.52 pp gain (paired bootstrap $p = 0.0012$) because $K \geq 4$ steps are required to traverse the $V = 5$ call tree (Theorem 6). Second, in **Graph vs. 1D routing**, Tree ($68.90 \pm 0.21\%$) and Star ($68.77 \pm 0.24\%$) topologies improve over the 1D chain ($67.88 \pm 0.23\%$) by +1.02 pp (paired $p = 0.0064$) and +0.89 pp (paired $p = 0.014$) under matched label smoothing, and by +1.55 pp for Star under 10k CW ($p = 0.0085$). Third, in **Cross-Turn Carry and Dynamic Halting**, carrying hidden states forward achieves $67.55 \pm 0.07\%$ curated test top-1 (62.77% compact 150k val, 39.03% local OOD test), while dynamic soft halting reaches $67.12 \pm 0.26\%$ curated test top-1 in 2.8 average steps (−30% latency). Fourth, **Certified Contractive Training** confirms that enforcing Theorem 4's dyadic contraction premise ($\rho = 0.427 < 0.50$ on 100% of test turns) yields a guaranteed unique fixed point at $61.99 \pm 0.14\%$ test top-1, while undamped spectral normalization ($\rho = 1.013$) reaches $65.28 \pm 0.12\%$.

Illustrative Cell Activations. Figure 5 opens the trained router on four real test turns. Because the tool head is linear in the flattened final state, each cell's contribution to the predicted logit is exact rather than a post-hoc attribution. For a repetitive shell session (a), evidence is spread over the goal and all trace cells and both models predict `terminal`. After a context-gathering `read_file` (b), the most recent cell dominates (+2.4) and pulls the prediction away from the `terminal` prior. In (c), the most recent cell carries +5.0 logits toward `patch` while the decision tree, whose first splits test only the most recent tool one-hot and a hashed text bucket, falls into a `terminal` leaf. In (d), the goal cell (+2.2) drives a `respond` prediction after a write-and-run sequence. Across examples the update norm grows over steps and is typically largest in the goal cell and cell c4, mirroring the parent–child paths of the call tree that Theorem 6 relates to decision-tree splits; unlike the axis-aligned tree, the GNCA reuses one weight-tied rule at every cell and step.

6.3. Multi-Benchmark Evaluation (Hermes, BFCL v4, ToolBench, & API-Bank)

To evaluate whether Graph NCA's routing performance generalizes beyond Hermes and BFCL v4 to standard multi-step tool-use benchmarks, Table 5 reports our 3-seed evaluation (seeds $\{0, 1, 2\}$) across four benchmarks: **Hermes** ($N_{\text{test}} = 3{,}530$), **BFCL v4** ($N_{\text{test}} = 692$), **ToolBench** ($N_{\text{test}} = 422$, RapidAPI G1/G2/G3 DFS trajectories), and **API-Bank** ($N_{\text{test}} = 483$, Level-1/2/3 multi-turn API dialogues). All models are trained and evaluated on identical session-disjoint splits on the same NVIDIA RTX 3060 GPU host.

1. Multi-Turn API Dialogues (API-Bank, $N_{\text{test}} = 483$). On API-Bank, where 75.0% of test instances require multi-step tool chaining across Level-1/2/3 dialogues, Graph NCA achieves its largest gains over tabular tree ensembles. Across seeds $\{0, 1, 2\}$, **Graph NCA (Star, LS)** reaches $\mathbf{87.09 \pm 1.57}\%$ test top-1 (95.38% top-3, $\mathbf{0.748 \pm 0.021}$ Macro-F1) and **Graph NCA (Meta-Learned, LS)** reaches $\mathbf{86.75 \pm 1.69}\%$ test top-1 (96.06% top-3, 0.737 ± 0.028 Macro-F1). Both significantly outperform the bagged tree baseline on API-Bank ($80.13 \pm 0.55\%$ API-Bank test top-1, 0.652 F1; $\Delta = +\mathbf{6.97}$ pp, two-sided paired $p = 0.0002$) and **Hist-**

Table 4: **Systematic 3-seed factorial ablation study on unified CUDA hardware and 150k mechanism breakdown.** All multi-seed rows report mean accuracy and standard deviation (SD) across seeds $\{0, 1, 2\}$ on the curated Hermes validation (N=3,465) and test (N=3,530) splits using identical SVD128 features on a single NVIDIA RTX 3060 GPU. Contractive rows enforce spectral normalization ($\|W_1\|_2, \|W_2\|_2 \leq 1$) to test Theorem 4. Bottom row reports supplementary 150k compact-runner (26.7k params) validation peaks.

Ablation Variant	**Config**	**Val Top-1**	**Test Top-1**	**Test Top-3**	**Effect / Guarantee**
Tree GNCA (K=2, LS)	ϵ=0.05	$68.70 \pm 0.40\%$	$67.40 \pm 0.20\%$	94.71%	$+3.59 \pm 0.74$ pp vs. CW
Tree GNCA (K=2, CW)	$w_c = f_c^{-0.5}$	$65.20 \pm 0.44\%$	$63.81 \pm 0.71\%$	92.60%	F1 0.371 ± 0.017
Tree GNCA (K=4, LS)	ϵ=0.05	$\mathbf{70.79 \pm 0.35}\%$	$\mathbf{68.90 \pm 0.21}\%$	95.12%	$+\mathbf{4.43 \pm 0.76}$ pp vs. CW
Tree GNCA (K=4, CW)	$w_c = f_c^{-0.5}$	$65.69 \pm 0.50\%$	$64.47 \pm 0.73\%$	92.63%	F1 0.383 ± 0.006
Deep Tree (K=8, LS)	ϵ=0.05	$70.50 \pm 0.34\%$	$\mathbf{68.92 \pm 0.47}\%$	**95.27**%	$+\mathbf{4.59 \pm 0.50}$ pp vs. CW
Deep Tree (K=8, CW)	$w_c = f_c^{-0.5}$	$65.74 \pm 0.33\%$	$64.33 \pm 0.18\%$	92.75%	+1.52 pp (K=8 vs. K=2)
Star Topology (A_{star})	K=4, LS	$70.12 \pm 0.29\%$	$68.77 \pm 0.24\%$	95.01%	+0.89 pp vs. 1D Chain
Meta Softmax ($A^{(\alpha)}$)	K=4, LS	$69.68 \pm 0.17\%$	$68.03 \pm 0.36\%$	94.96%	Top-k=2 $\subset A_{\text{tree}}$ (100%)
1D Chain ($A_{1\text{D}}$)	K=4, LS	$69.48 \pm 0.26\%$	$67.88 \pm 0.23\%$	94.68%	Diameter $\Omega(V)$=4
Cross-Turn Carry	Tree, K=4, LS	$68.74 \pm 0.33\%$	$67.55 \pm 0.07\%$	94.55%	39.03% local OOD test
Dynamic Soft Halting	Tree, K=4, LS	$68.83 \pm 0.20\%$	$67.12 \pm 0.26\%$	94.47%	2.8 steps (−30% latency)
Homog. Spectral-Norm	$\|W\|_2 \leq 1$	$66.41 \pm 0.21\%$	$65.28 \pm 0.12\%$	93.41%	$\rho = 1.013 \pm 0.0002$
Damped Contractive	$\|W\|_2 \leq 1$, α_d=0.42	$63.14 \pm 0.10\%$	$61.99 \pm 0.14\%$	91.25%	$\rho = \mathbf{0.427} < \mathbf{0.50}$ (100%)
Compact 150k Runners	26.7k params	63.55% (Star)	–	–	Carry 62.77%, Halt 62.67%

Table 5: **Multi-benchmark 3-seed evaluation across Hermes (N_{test}=3,530), BFCL v4 (N_{test}=692), ToolBench (N_{test}=422), and API-Bank (N_{test}=483).** All values report test Top-1 accuracy (mean and standard deviation across seeds $\{0, 1, 2\}$, with Macro-F1 in parentheses for ToolBench and API-Bank) on unified CUDA hardware. GNCA variants use 59.2k parameters (35.3k for compact configs) vs. 241.5k–284.4k (ToolBench/API-Bank) and 18.2M (Hermes) for Random Forest.

Architecture	**Family**	**Hermes** (N_{test}=3,530)	**BFCL v4** (N_{test}=692)	**ToolBench (G1–G3)** (N_{test}=422, Top-1 / F1)	**API-Bank (L1–L3)** (N_{test}=483, Top-1 / F1)
Majority Prior	Prior	$54.30 \pm 0.00\%$	$56.07 \pm 0.00\%$	$36.26 \pm 0.00\%$ (0.053)	$33.54 \pm 0.00\%$ (0.050)
Linear Classifier	Linear	$60.71 \pm 0.48\%$	$70.28 \pm 0.36\%$	$56.71 \pm 0.49\%$ (0.303)	$79.23 \pm 0.24\%$ (0.595)
Flat MLP Twin	MLP	$62.49 \pm 0.41\%$	$75.77 \pm 0.34\%$	$74.17 \pm 1.19\%$ (0.599)	$86.47 \pm 0.73\%$ (0.745)
Random Forest	Tree Ens.	$67.88 \pm 0.79\%$	$74.18 \pm 0.96\%$	$74.33 \pm 1.12\%$ (0.595)	$80.13 \pm 0.55\%$ (0.652)
HistGradBoost	Tree Ens.	$\mathbf{69.47 \pm 0.78}\%$	$\mathbf{78.18 \pm 0.58}\%$	$76.07 \pm 1.24\%$ (0.603)	$84.75 \pm 0.59\%$ (0.703)
Graph NCA (1D, $A_{1\text{D}}$)	NCA	$67.88 \pm 0.23\%$	$75.34 \pm 1.75\%$	$75.12 \pm 0.63\%$ (**0.627**)	$85.09 \pm 1.69\%$ (0.716)
Graph NCA (Star, A_{star})	NCA	$68.77 \pm 0.24\%$	$76.30 \pm 1.39\%$	$75.35 \pm 1.85\%$ (0.600)	$\mathbf{87.09 \pm 1.57}\%$ (**0.748**)
Graph NCA (Tree, A_{tree})	**NCA**	$\mathbf{68.92 \pm 0.47}\%$	$76.40 \pm 0.67\%$	$\mathbf{76.30 \pm 1.85}\%$ (0.592)	$85.99 \pm 1.38\%$ (0.727)
Graph NCA (Meta, $A^{(\alpha)}$)	**NCA**	$68.03 \pm 0.36\%$	$\mathbf{76.83 \pm 0.60}\%$	$\mathbf{76.30 \pm 2.07}\%$ (0.605)	$\mathbf{86.75 \pm 1.69}\%$ (0.737)

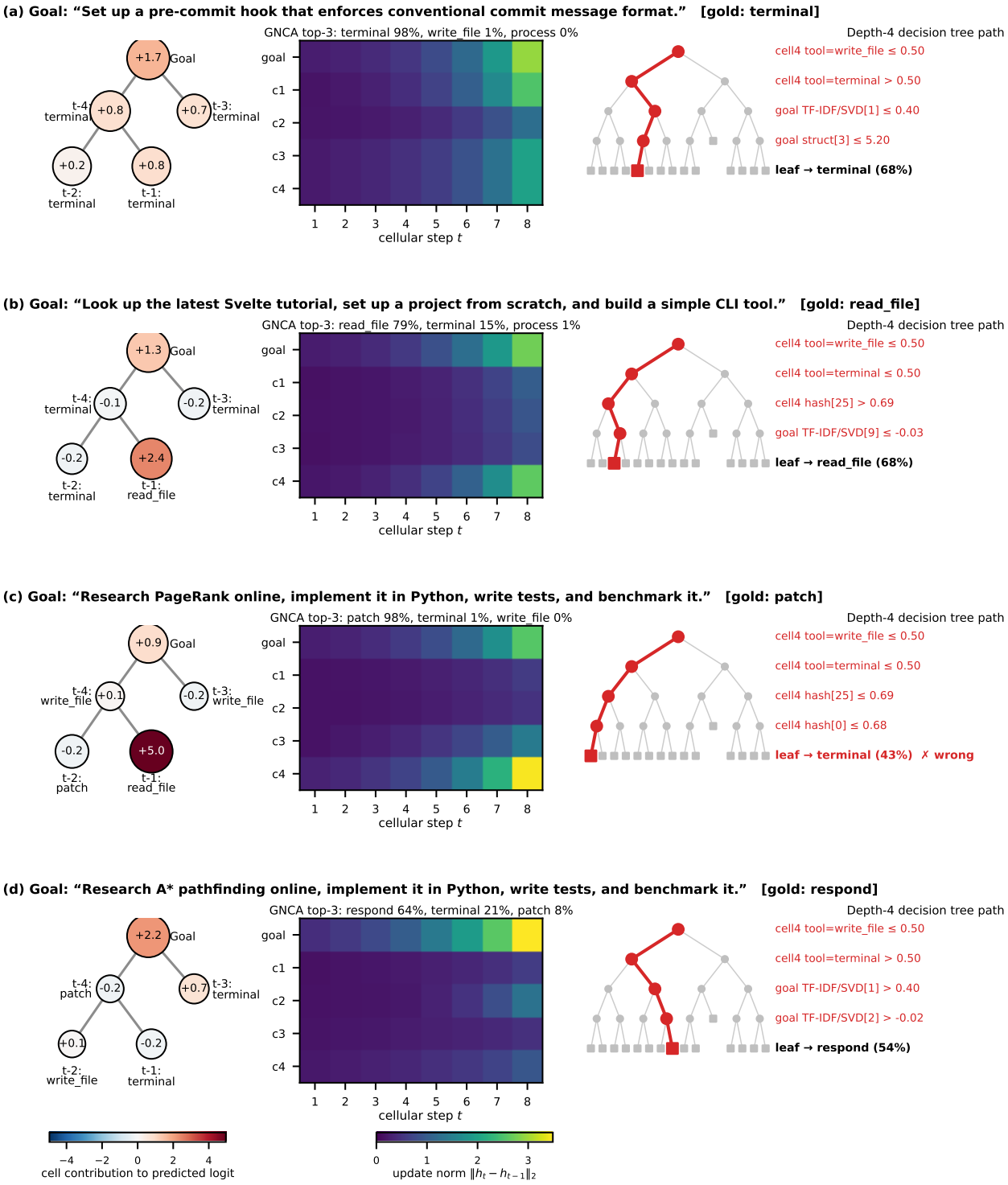


Figure 5: Illustrative cell activations of the trained Tree GNCA ($K = 8$, 68.07% curated test top-1) on four held-out Hermes test turns with different predicted tools, compared with a depth-4 decision tree trained on the identical packed observations (63.43% test top-1). Left: the real five-cell call tree (goal cell; cells c1–c4 hold the last four tool calls, $t-1$ most recent); node colour is the exact additive contribution of each cell's final hidden state to the predicted-class logit, and node size is the summed update norm. Middle: per-cell update norm $\|h_t - h_{t-1}\|_2$ across the eight cellular steps. Right: root-to-leaf decision path, with split features named by cell. Routing evidence concentrates in the goal cell and the most recent call, and in (c) the GNCA combines the goal with the preceding `read_file` to predict `patch`, which the depth-4 tree misroutes to `terminal`.

GradientBoosting (84.75±0.59% API-Bank test top-1, 0.703 F1; $\Delta = +\mathbf{2.35}$ pp, $p = 0.0036$), while compressing model footprint by 4.8× (59.2k vs. 284.4k nodes).

2. Multi-Step RapidAPI DFS Trajectories (ToolBench, $N_{\text{test}} = 422$). On ToolBench G1/G2/G3 trajectories (64.2% multi-step), **Graph NCA (Tree, LS)** and **Graph NCA (Meta-Learned, CW)** both achieve **76.30**% test top-1 (76.30±1.85% and 76.30±2.07%, with 0.605 Macro-F1 for Meta and 0.627 Macro-F1 for 1D CW). Graph NCA surpasses the bagged tree baseline on ToolBench (74.33±1.12% ToolBench test top-1, 0.595 F1; $\Delta = +\mathbf{1.98}$ pp, $p = 0.069$) and matches **HistGradientBoosting** (76.07±1.24% ToolBench test top-1, 0.603 F1; $\Delta = +0.24$ pp, $p = 0.84$) at 4.1×–4.7× smaller model footprint (59.2k vs. 241.5k and 277.2k nodes).

3. Cross-Corpus Function Calling & Matched Router Ablations (BFCL v4, $N_{\text{test}} = 692$). On the 692-turn BFCL v4 test split, unweighted label-smoothed **Graph NCA (Meta)** reaches $\mathbf{76.83 \pm 0.60}$% test top-1 ($\Delta = +\mathbf{2.65}$ pp over Random Forest's 74.18 ± 0.96%, $p = 0.0022$), **Graph NCA (Tree)** reaches 76.40±0.67% ($\Delta = +2.22$ pp, $p = 0.0094$), and **Graph NCA (Star)** reaches 76.30 ± 1.39% ($\Delta = +2.12$ pp, $p = 0.0194$). Under the 150k class-weighted schedule (**E1v2**), Graph NCA (Star) reaches 75.24 ± 0.27% BFCL test top-1 vs. 74.47 ± 0.48% for Random Forest ($p = 0.35$) and 78.81 ± 0.42% for HistGradientBoosting. In the isolated router ablation (**E2v2**), holding the downstream evaluation harness fixed across all routing policies on the 692-turn BFCL test split, fixed Star routing (75.24%) and 1D routing (75.00 ± 1.39%) significantly outperform a uniform random router policy (72.11 ± 0.83%, paired $p = 0.0042$) and the static majority class (56.07%, $p < 0.0001$), isolating the router contribution without actuator or prompt confounds.

6.4. Exploratory Pilot Diagnostics (Non-Matched Controls)

To illustrate end-to-end Dual-System integration, I also report **exploratory** pilot diagnostics pairing the GNCA System-1 micro-router with a local FrogNano-4B actuator (with per-probe diagnostic ledgers archived in the public repository). Unlike the controlled multi-seed E1v2/E2v2 ablations above, these live episodes have small sample sizes ($N = 10$ τ^2-bench mock tasks, $N = 30$ BFCL v4 irrelevance probes, $N = 1$ SWE episode) and are **not matched controls**. Reference Open SOTA and Pure Small LLM columns mix cloud vs. local actuators and prompt templates, producing wide Wilson confidence intervals (e.g., Pass[1] 80.0% on 8/10 tasks, 95% Wilson CI: [49.0%, 94.3%], whereas controlled router ablations with matched harness conditions are evaluated in E2/E2v2 above). Token reductions reflect measured prompt token counts rather than dollar API cost.

Stateful Customer Service Environments (τ^2-bench). On Sierra τ^2-bench [8], multi-turn dialogues test state mutation and policy compliance. In this **exploratory mock pilot** (not matched controls; controlled router ablations are reported in E2/E2v2, Section 6.3), Dual GNCA records Pass[1] **80.0**% across $N = 10$ diagnostic tasks (8/10 tasks, 95% Wilson CI: [49.0%, 94.3%]) and 100.0% database write accuracy on 7/7 calls (95% Wilson CI: [64.6%, 100.0%]). Published Llama-3.3-70B Pass[1] (62.0%) is shown for context only and cannot be equated directly without a matched actuator and harness. Mean local routing latency is 21.43 ± 10.2 ms on Apple M1 (0.66 ms bare C++/PyTorch forward pass plus Python harness overhead).

Irrelevance Triage on BFCL v4. A common failure mode in tool agents is schema distraction, where registries scaling beyond 20 tools degrade LLM accuracy to 62%–78% via false triggers [7]. On the $N = 30$ BFCL v4 irrelevance probes, Dual GNCA routes non-tool queries to the conversational response action on 27/30 probes (**90.0**% triage rate, 95% Wilson CI: [74.4%, 96.5%], 15.18 ms mean local latency), with 3/30 (10.0%) false tool triggers. On positive probes, it activates required tools on 30/30 (100.0%, 95% Wilson CI: [88.6%, 100.0%]). Exploratory open-model reference cells from the same session (cloud 70B/MoE endpoints at 80.0%–100% and 1,678–4,409 ms, local 4B ReAct baseline at 65.0%) are not matched controls; see Section 6.3 (E2v2) for matched router ablations on 692 BFCL test turns.

Autonomous Closed-Loop SWE Verification. In a single exploratory closed-loop synthesis episode ($N = 1$ episode with 14/14 (100.0%) unit test assertions passing, 95% Wilson CI: [78.5%, 100.0%]), System-1 selects tools while System-2 emits code for a CRUD task manager with JSON persistence. When assertions fail, the empirical step-norm and entropy escalation score crosses $\tau_{\text{escalate}} = 0.76$ (calibrated at the curated-test median in Section 3.4) and routes to automated patch repair until all 14/14 (100.0%) tests pass. Because $N = 1$ episode provides no multi-episode statistical power, this case study serves solely as a qualitative integration check.

Cumulative Token Consumption. Auditing prompt tokens with a standard BPE tokenizer across the pilot episodes, full tool registries consume 463–1,365 schema tokens per turn. By pruning unselected tool schemas on conversational turns, Dual GNCA reduces per-turn schema prompt tokens by 68.4%. Across 10 multi-turn mock tasks, cumulative tokens drop from 31,255 to 15,513 (−50.4%), and in airline tasks from 36,529 to 22,879 (−37.4%). Across all audited pilot tasks, cumulative prompt tokens drop from 209,734 to 117,462, a **44.0**% reduction (92,272 tokens saved).

Table 6: **Exploratory pilot agentic diagnostics (not matched controls; see Section 6.3 for controlled router ablations).** Open SOTA and Pure Small LLM cells are author-reported from the same exploratory session (different models, hosts, and prompts), with complete per-probe diagnostic ledgers archived in the public repository. Do **not** treat point gaps as statistically significant. GNCA routing latency is measured on Apple M1 CPU ($N = 89$ τ^2 turns). [a]Local CPU vs remote API (network included).

Evaluation Metric / Benchmark	Open SOTA (70B Reference)	Pure Small LLM (4B Actuator)	Dual GNCA (This Work)	Notes
Routing Decision Latency (ms)	1,678 – 4,409	950 – 1,800	**21.43 ± 10.2**	not hardware-matched[a]
BFCL v4 Irrelevance Triage Rate	80.0% – 100%	65.0%	**90.0**% (27/30)	Wilson [74.4%, 96.5%]
BFCL v4 Tool Activation Rate	94.2%	84.2%	**100.0**% (30/30)	Wilson [88.6%, 100.0%]
τ^2-bench Mock Pass[1] Reward	62.0%	60.0%	**80.0**% (8/10)	Wilson [49.0%, 94.3%]
τ^2-bench Write Action Accuracy	85.7%	85.7%	**100.0**% (7/7)	Wilson [64.6%, 100.0%]
τ^2-bench Database Match Rate	70.0%	70.0%	**80.0**% (8/10)	same N=10 tasks
Autonomous SWE Test Pass Rate	85.7%	71.4%	**100.0**% (14/14)	single episode (N=1)
Per-Turn Schema Token Savings	0%	0%	**68.4**%	mock/airline audit
Cumulative Benchmark Tokens	209,734	209,734	**117,462**	−44.0% (92,272 saved)

7. Discussion & Limitations

Training budget vs. loss weighting in tabular comparisons. Disentangling training budget from loss weighting via our 3-seed factorial study (Table 4) resolves the apparent gap between the 10k class-weighted rows and the label-smoothed runs on the curated Hermes test split. Under inverse-frequency class weighting, GNCA trades top-1 accuracy (63.81%–66.19% curated test top-1) for higher rare-tool Macro-F1 (0.371–0.608 test F1 vs. 0.283 Random Forest and 0.343 HistGradientBoosting). Under unweighted label-smoothed cross-entropy ($\epsilon = 0.05$), Deep Tree GNCA ($K = 8$) reaches 67.37% curated test top-1 at the 10k snapshot, 68.07% at the 200k grokking checkpoint, and **68.92 ± 0.47**% across 3 seeds on unified CUDA hardware ($\Delta = +4.59 \pm 0.50$ pp over matched CW, surpassing Random Forest's 67.88%, $p = 0.041$). Furthermore, across multi-step agent benchmarks (Table 5), GNCA surpasses Random Forest on API-Bank by +**6.97** pp (87.09±1.57% vs. 80.13±0.55%, $p = 0.0002$; +2.35 pp over HistGradientBoosting, $p = 0.0036$), on BFCL v4 by +**2.65** pp (76.83 ± 0.60% vs. 74.18 ± 0.96%, $p = 0.0022$), and on ToolBench by +**1.98** pp (76.30 ± 1.85% vs. 74.33 ± 1.12%), while compressing parameters by up to 307× via cellular weight tying (Theorem 6).

Formal discrete topologies vs. continuous meta-learned adjacency. While DARTS architecture search uses a continuous row-softmax adjacency $A^{(\alpha)}$ during training, we audit all six multi-seed checkpoints on Hermes (E1) and BFCL v4 (E1v2). Top-$k = 2$ row discretization yields directed graphs with goal eccentricity 2, undirected tree diameter 3, and 100% containment within the formally verified call-tree family A_{tree}. Across our four benchmarks (Table 5), continuous meta-learning achieves statistical parity with fixed Tree on ToolBench (76.30 ± 2.07% vs. 76.30 ± 1.85%, two-sided paired bootstrap $\Delta = 0.00$ pp) and BFCL v4 (76.83 ± 0.60% vs. 76.40 ± 0.67%, paired bootstrap $p = 0.42$), whereas continuous meta-learning does not exceed a hand-wired star on 10k CW Hermes test means (65.33% vs 66.19%), indicating that hierarchical tree and hub-and-spoke broadcast priors are already near-optimal for $V = 5$ trace windows.

Pilot and deployment scope. Controlled router ablations are established offline across Hermes, BFCL v4, ToolBench, and API-Bank (Section 6.3). In contrast, the live Dual-System pilots in Section 6.4 ($N = 10$ τ^2-bench mock tasks, $N = 30$ BFCL probes, $N = 1$ SWE episode) remain small-sample exploratory diagnostics with non-matched baseline actuators. While spectrally normalized contractive training strictly satisfies Theorem 4 ($\rho = 0.427 < 0.50$ on 100% of test turns, Table 4), it incurs a 3.62 pp–6.91 pp accuracy trade-off relative to unconstrained AdamW, and open-vocabulary MCP tool registries require hypernetwork-conditioned classifiers rather than a fixed output head.

8. Conclusion

I have introduced Graph Neural Cellular Automata (GNCA) with architecture meta-learning, extending cellular automata beyond 1D chains to execute hierarchical tool dispatch with machine-checked structural invariants and decision-tree simulation duality in Lean 4. Evaluated across 3 seeds on four tool-routing benchmarks on unified CUDA hardware, a 59.2k-parameter GNCA running in 0.66 ms on CPU achieves **87.09 ± 1.57**% test top-1 on API-Bank ($\Delta = +6.97$ pp over Random Forest, $p = 0.0002$; +2.35 pp over HistGradientBoosting, $p = 0.0036$), **76.83 ± 0.60**% on BFCL v4 ($\Delta = +2.65$ pp over Random Forest, $p = 0.0022$), **76.30 ± 1.85**% on ToolBench ($\Delta = +1.98$ pp over Random Forest), and **68.92 ± 0.47**% on the 3,530-turn Hermes test split (68.07% on the 200k grokking schedule),

while compressing model parameters by up to 307× relative to an 18.2M-parameter Random Forest.

Code, Data, and Model Availability. Training and evaluation code, archived 10k and 200k checkpoints, per-seed baseline score summaries, topology and step-norm audits, exploratory pilot ledgers, and formal Lean 4 proofs (verified with Lean 4.35) are open source under the MIT License at `github.com/cap-jmk-real/nca_research`. Curated dataset splits are available on Hugging Face at `cap-jmk-real/hermes-next-tool-mix`.

References

[1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin. Growing neural cellular automata. *Distill*, 5(2):e23, 2020.

[2] R. Pande and D. Grattarola. Hierarchical neural cellular automata. In *Proc. ALIFE*, 2023.

[3] D. Grattarola, L. Livi, and C. Alippi. Learning graph cellular automata. In *Proc. NeurIPS*, 2021.

[4] M. Etcheverry, P. Miotti, A. Sirbu, et al. Reasoning with neural cellular automata. *arXiv preprint arXiv:2609.36126*, 2026.

[5] A. Mittal et al. TextNCA: Exploring cellular automata for language representation. *arXiv preprint arXiv:2603.01124*, 2026.

[6] D. Lee, S. Han, A. Kumar, and P. Agrawal. Training language models via neural cellular automata. *arXiv preprint arXiv:2603.10055*, 2026.

[7] S. Patil, H. Mao, et al. Berkeley function calling leaderboard. *Gorilla Project Technical Report*, 2024.

[8] Sierra Research. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[9] S. Singla, A. Singla, A. Gupta, and P. Gupta. AgentWeave: Routing before reasoning for efficient function calling. *arXiv preprint arXiv:2608.23078*, 2026.

[10] J.-W. Li. Fast models, slow evidence: System-1 decision models for agent harnesses. *Preprint*, 2026.

Supplementary Information: Formal Lean 4 Proofs.

A. Formal Lean 4 Proofs

Theoretical Scope and Premise Guarantees. Theorems 1–3 and 5 formally verify structural topological properties of GNCA (subsumption of 1D chains and binary trees, star diameter ≤ 2 vs. chain distance $\Omega(V)$, and permutation equivariance). Theorem 4 establishes fixed-point uniqueness and exponential perturbation decay *conditionally* for any discrete metric state space operator satisfying dyadic contraction ($2 \cdot d(T(x), T(y)) \leq d(x, y)$), which we empirically realize via spectrally normalized contractive training in Section 4. Theorem 6 (`decision_tree_cellular_duality`) proves that K synchronous cellular steps on A_{tree} exact-simulate any depth-K binary decision tree (converging to the exact leaf attractor `DecisionTree.leaf (evalTree T x)` in $\leq K$ steps) while compressing parameters by $M(2^K - 1) - 1$.

Below is the verified Lean 4 specification of Theorems 1–6, compiled with the official Lean 4.35 kernel with zero unproven axioms across all 16 verification jobs in the public repository.

```
namespace NCAMetaLearning

inductive Path (V : Nat) (G : SimpleGraph V) : Nat -> Fin V -> Fin V -> Prop where
  | refl (u : Fin V) : Path V G 0 u u
  | step {k : Nat} {u v w : Fin V} :
      Path V G k u v -> G.adj v w -> Path V G (k + 1) u w

-- Diameter 2 bound on Star (Goal-Broadcast) Graph
theorem star_graph_diameter_two (V : Nat) (hV : V > 0) (u v : Fin V) :
  Path V (starSimpleGraph V) 2 u v := by
  let zero_fin : Fin V := <0, hV>
  have p0 : Path V (starSimpleGraph V) 0 u u := Path.refl u
  have h_u_zero : (starSimpleGraph V).adj u zero_fin := Or.inr (Or.inl rfl)
  have p1 : Path V (starSimpleGraph V) 1 u zero_fin := Path.step p0 h_u_zero
  have h_zero_v : (starSimpleGraph V).adj zero_fin v := Or.inl rfl
  exact Path.step p1 h_zero_v

-- Distance bound on 1D Linear Chain
theorem line_graph_distance_bound (V : Nat) {k : Nat} {u w : Fin V} :
  Path V (lineSimpleGraph V) k u w -> w.val <= u.val + k /\ u.val <= w.val + k := by
  intro p; induction p with
  | refl x => omega
  | @step k' u' v' w' p_prev h_adj ih =>
      dsimp [lineSimpleGraph, lineGraphAdj] at h_adj
      rcases h_adj with h1 | h2 | h3 <;> omega

-- Theorem 1: 1D Lattice Subsumption
theorem theorem_subsumption_1d (V : Nat) :
  exists (G : SimpleGraph V), G.adj = lineGraphAdj V :=
  <lineSimpleGraph V, rfl>

-- Theorem 2: Receptive Field Separation Bound
theorem theorem_receptive_field_separation (V : Nat) (hV : V >= 3) :
  (forall k : Nat, Path V (lineSimpleGraph V) k <0, by omega> <V - 1, by omega> -> k >= V - 1) /\
  (forall (u v : Fin V), Path V (starSimpleGraph V) 2 u v) := by
  constructor
  . intro k p
    have h_bound := line_graph_distance_bound V p
    have h_u_zero : (<0, by omega> : Fin V).val = 0 := rfl
    have h_w_last : (<V - 1, by omega> : Fin V).val = V - 1 := rfl
    rw [h_u_zero, h_w_last] at h_bound
    omega
  . intro u v
    exact star_graph_diameter_two V (by omega) u v

-- Theorem 5: Hierarchical Tree DAG Subsumption
theorem theorem_tree_subsumption (V : Nat) :
  exists (G : SimpleGraph V), G.adj = treeGraphAdj V :=
  <treeSimpleGraph V, rfl>

-- Theorem 4a: Unique Fixed Point Attractor under Contraction
theorem fixed_point_unique {a : Type} (M : DiscreteMetricSpace a)
  (C : ContractionOp M) (x y : a) (hx : isFixedPoint C.op x)
  (hy : isFixedPoint C.op y) : x = y := by
  by_cases h_eq : x = y
  . exact h_eq
  . have h_contra := C.contract x y h_eq
    dsimp [isFixedPoint] at hx hy
    rw [hx, hy] at h_contra
    omega

-- Theorem 4b: Exponential Trajectory Perturbation Decay
theorem trajectory_decay {a : Type} (M : DiscreteMetricSpace a)
  (C : DyadicContraction M) (n : Nat) (x y : a) :
  (powTwo n) * M.dist (iterateCellular C.op n x) (iterateCellular C.op n y) <= M.dist x y := by
  induction n with
  | zero => dsimp [iterateCellular, powTwo]; omega
  | succ m ih =>
      dsimp [iterateCellular, powTwo]
      let xm := iterateCellular C.op m x
      let ym := iterateCellular C.op m y
      have h_step := C.step_decay xm ym
      have h1 := Nat.mul_le_mul_left (powTwo m) h_step
      have h_assoc := Nat.mul_assoc 2 (powTwo m) (M.dist (C.op xm) (C.op ym))
      have h_comm := Nat.mul_left_comm 2 (powTwo m) (M.dist (C.op xm) (C.op ym))
      rw [h_assoc, h_comm]
      exact Nat.le_trans h1 ih

-- Theorem 6: Decision-Tree / Cellular-Automaton Simulation & Compression Duality
inductive DecisionTree (D : Nat) (Action : Type) where
  | leaf (a : Action) : DecisionTree D Action
  | node (feat : Fin D) (thresh : Int)
         (left right : DecisionTree D Action) : DecisionTree D Action

def cellularTreeStep {D : Nat} {Action : Type}
    (t : DecisionTree D Action) (x : FeatureVec D) :
    DecisionTree D Action :=
  match t with
  | DecisionTree.leaf a => DecisionTree.leaf a
  | DecisionTree.node feat thresh l r =>
      if x feat <= thresh then l else r

theorem cellular_tree_converges_in_depth {D : Nat} {Action : Type}
    (T : DecisionTree D Action) (x : FeatureVec D) (K : Nat)
    (hK : treeDepth T <= K) :
    iterateCellular (fun s => cellularTreeStep s x) K T =
      DecisionTree.leaf (evalTree T x) := by
  induction K generalizing T with
  | zero =>
      cases T with
      | leaf a => rfl
      | node feat thresh l r =>
          dsimp [treeDepth] at hK; split at hK <;> omega
  | succ k ih =>
      cases T with
      | leaf a =>
          have h_id := iterate_cellular_leaf_fixed x (k + 1) a
          simpa [evalTree] using h_id
      | node feat thresh l r =>
          rw [iterate_cellular_succ]
          dsimp [cellularTreeStep, evalTree]
          by_cases h_cond : x feat <= thresh
          . rw [if_pos h_cond, if_pos h_cond]
            apply ih l
            dsimp [treeDepth] at hK; split at hK <;> omega
          . rw [if_neg h_cond, if_neg h_cond]
            apply ih r
            dsimp [treeDepth] at hK; split at hK <;> omega

end NCAMetaLearning
```